UNIVERSITÀ DI PISA



Dipartimento di Informatica

Corso di Laurea in Informatica

Implementazione Python di un Decision Tree Categorico Multi-Split

Relatore:

Prof. Riccardo Guidotti

Presentata da:

Giuseppe Muschetta

Anno Accademico 2022/2023

A me stesso,

per essere tornato a crederci nonostante l'età.

Indice

1 Introduzione **4**

2 Background **7**

2.1 Concetti di Base 7

2.2 Decision Tree Classifier 10

2.3 L'algoritmo di Hunt 14

2.4 Misure di Impurità 17

2.4.1 Misure di Impurità per un nodo singolo 17

2.4.2 Impurità post parent split dei nodi figli 19

2.5 Caratteristiche finali dei Decision Tree 20

3 Implementazione **22**

3.1 Classe interna Node e Classe MixedDecisionTree 24

3.2 Impurità e calcolo del Gain 26

3.3 Il Nucleo di tutto: L'algoritmo ID3 27

3.3.1 Criteri Di Stop 28

3.3.2 Criteri di Pre-Pruning 29

3.4 Gestione degli Attributi Continui e Categorici 30

3.4.1 Gestione attributi nel metodo fit 31

3.4.2 Gestione attributi nel metodo predict 32

3.5 Trasformazione degli attributi continui in categorici 35

3.6 Visualizzazione del MDT 37

3.7 Le Tre Modalità del MixedDecisionTree 39

3.7.1 Modalità Classic 40
3.7.2 Modalità Median 41
3.7.3 Modalità Binned 42

4 Esperimenti **44**
4.1 Informazioni preliminari 44
4.2 Presentazione dei Dataset 45
4.3 Test 46
4.3.1 Metrica considerata: ACCURACY 48
4.3.2 Metrica considerata: F1_SCORE 59
4.3.3 Misuriamo il tempo 66

5 Conclusioni **68**

A **70**

Capitolo 1

Introduzione

> *To me, mathematics, computer science, and the arts are insanely related. They're all creative expressions. (Sebastian Thrun)*

In questa tesi, è stato implementato da zero un albero decisionale utilizzando il linguaggio di programmazione Python. La scelta del Python è motivata dalla presenza di librerie avanzate, come Numpy e Pandas, che facilitano lo sviluppo di software per manipolare array, liste e dataset. Nei paragrafi successivi, approfondiremo il concetto di dataset e, nel terzo capitolo, vedremo come essi vengono manipolati tramite la libreria Pandas.

L'apprendimento tramite alberi decisionali è un approccio supervisionato utilizzato in Statistica, Data Mining e Machine Learning, noto anche come apprendimento automatico. Questo metodo permette alle macchine di apprendere dai dati in maniera autonoma e senza istruzioni esplicite, in quanto i dati stessi definiscono l'apprendimento.

Nel contesto del machine learning, un albero decisionale è un modello predittivo che offre ottime prestazioni, funziona bene su dati tabellari e viene costruito utilizzando tecniche di apprendimento basate sui dataset iniziali. Solitamente, un dataset viene suddiviso in training set, utilizzato per creare la struttura dell'albero, e test set, impiegato per valutare l'accuratezza del modello predittivo. Le proporzioni di suddivisione possono essere gestite da una funzione di libreria, presentata nel terzo capitolo.

Gli alberi di decisione sono apprezzati per la loro facilità di lettura e interpretazione da parte degli esseri umani e per il loro costo computazionale ottimale. Una volta addestrato, un

albero di decisione funziona con una complessità temporale lineare rispetto alla profondità dell'albero stesso.

Il motivo per il quale si è scelto di voler creare ex novo un decision tree è dovuto al fatto che gli alberi decisionali presenti nella libreria scikit-learn sono progettati principalmente per lavorare con variabili numeriche, e non supportano *nativamente* le variabili categoriche. È necessario convertire le variabili categoriche in numeriche utilizzando tecniche come il one-hot encoding prima di addestrare l'albero.

Un'ulteriore mancanza della libreria è il supporto per il multi-split: gli alberi decisionali in scikit-learn supportano solo split binari. In altre parole, ogni nodo radice o interno dell'albero ha esattamente due figli. Questo può essere limitante per alcuni problemi in cui uno split multi-way potrebbe essere più appropriato o efficiente.

Nella presente tesi, viene implementato un albero decisionale che è **categorico** e supporta il **multi-split**, consentendo una migliore leggibilità, versatilità e aderenza alla letteratura sugli alberi decisionali. Invece, come detto sopra, l'albero decisionale fornito dalla nota libreria Python scikit-learn, sebbene sia computazionalmente efficiente, presenta alcune limitazioni importanti. In particolare, non supporta il multi-split e non è progettato per lavorare nativamente con variabili categoriche. Tali restrizioni possono ridurre la flessibilità e l'applicabilità dell'approccio scikit-learn in alcuni contesti.

Si è scelto di denominare l'albero decisionale qui proposto **MixedDecisionTree**, dal momento che legge sia valori categorici (nativamente) che continui (tramite modifiche esterne all'albero) e permette un numero arbitrario di figli per ogni nodo, a differenza del DT di sklearn.

Confronteremo i due alberi utilizzando metriche come *Accuracy* e *F1-score*, analizzando l'andamento delle stesse al variare di vari iper-parametri di pre-pruning come *max-depth*, *min-sample-split* e *min-sample-leaf* descritti in seguito più dettagliatamente.

Il codice del progetto è disponibile all'indirizzo:

https://github.com/peppe212/MixedDecisionTree

Nel secondo capitolo si presenteranno gli alberi decisionali e le basi teoriche che ne governano il funzionamento.

Il terzo capitolo porrà l'accento sugli elementi chiave dell'implementazione del MDT, introducendo due ulteriori modalità operative che si affiancheranno alla modalità predefinita, denominata *classic*: le modalità *median* e *binned*. Al fine di garantire confronti più approfonditi e significativi nel quarto capitolo, tali modalità verranno applicate anche all'albero decisionale di scikit-learn.

Nel quarto capitolo si procederà con l'esecuzione di una serie di test mirati alla creazione di tabelle, istogrammi e grafici sovrapposti, attraverso i quali si potranno mettere in evidenza i confronti più rilevanti tra il MDT e il DT di sklearn.

Infine, nel quinto ed ultimo capitolo si valuterà se l'obiettivo di sviluppare un DT versatile e di facile interpretazione, caratterizzato dalla gestione nativa di attributi categorici e dal pieno supporto al multi-split, sia stato conseguito con successo.

Capitolo 2

Background

> *Computer Science is no more about computers than astronomy is about telescopes.*
> *(Edsger Dijkstra)*

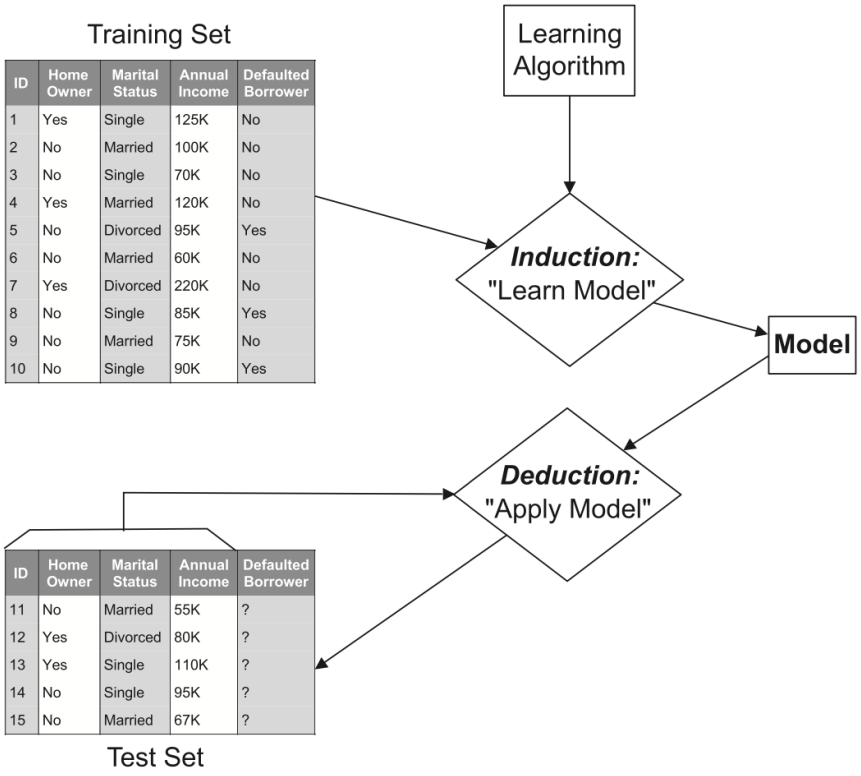
Questo capitolo pone il focus principalmente sugli aspetti teorici che riguardano gli alberi decisionali, si capirà come funzionano internamente, si studierà l'algoritmo di Hunt che è alla base di tutto e diverrà noto il meccanismo che rende effettivamente possibile l'apprendimento automatico, cioè il motivo per il quale un modello di classificazione di tipo decision tree riesce effettivamente ad *imparare* dai dati.

2.1 Concetti di Base

I Decision Tree sostanzialmente sono dei classificatori, e per classificazione si intende l'assegnamento di un'etichetta (di classificazione) a dati che non ne hanno una, cioè un classificatore conferisce loro una etichetta unica di classificazione.

Un **classificatore** è quindi il modello che esegue la classificazione. Molto spesso in letteratura si usano *classificatore* e *modello* come sinonimi.

Un classificatore è in genere descritto in termini di modello che viene creato utilizzando un determinato insieme di istanze, noto come training set, che contiene valori di attributi ed etichette di classe per ogni istanza. L'approccio sistematico per l'apprendimento di un modello di classificazione, dato un set di formazione, è noto come algoritmo di apprendimento.



ID	Home Owner	Marital Status	Annual Income	Defaulted Borrower
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

ID	Home Owner	Marital Status	Annual Income	Defaulted Borrower
11	No	Married	55K	?
12	Yes	Divorced	80K	?
13	Yes	Single	110K	?
14	No	Single	95K	?
15	No	Married	67K	?

Figura 2.1: Il ruolo della deduzione e dell'induzione [3]

Il processo di utilizzo di un algoritmo di apprendimento per costruire un modello di classificazione a partire dal training set è noto come induzione o bias induttivo dell'algoritmo. **L'induzione rende possibile l'apprendimento.** Questo processo di applicazione di un modello di classificazione su istanze di test nuove (mai riscontrate prima) per prevedere le loro etichette di classe è noto come **deduzione**. Pertanto, il processo di classificazione comporta due fasi:

1. applicare un algoritmo di apprendimento al training set affinché il modello "impari"
2. applicare il modello addestrato (che ha già imparato) per assegnare etichette a istanze sprovviste di etichetta, non etichettate.

Si sono menzionate parole come induzione e deduzione, esse esprimono i concetti chiave per costruire un modello di classificazione che apprende dai dati di training e poi riesce a predire etichette su dati mai riscontrati prima, o unseen. Nella Figura 2.1 è possibile visualizzare con chiarezza quanto appena detto: Le fasi di induzione e deduzione devono essere eseguite separatamente. *Il training set ed il test set devono essere indipendenti tra loro* per garantire che

il modello indotto possa prevedere con precisione le etichette di classe delle istanze che non ha mai incontrato prima. Si dice che i modelli che forniscono tali intuizioni predittive abbiano buone prestazioni di generalizzazione.

Le prestazioni di un modello (classificatore) possono essere valutate confrontando le etichette predette dall'albero con le etichette vere delle istanze di test. Con etichette vere delle istanze di test si intende che, quando viene effettuato il test, si conoscono già quali sono le etichette vere, altrimenti non avrebbe senso chiamarlo *test* set.

All'albero si danno in pasto tutte le istanze di test senza le etichette vere e si contano poi quante di esse vengono predette correttamente. Queste informazioni possono essere riassunte nella tabella seguente denominata *confusion matrix* per un problema di classificazione binario:

		Predicted Class	
		Class = 1	Class = 0
Actual Class	Class = 1	f_{11}	f_{10}
	Class = 0	f_{01}	f_{00}

Figura 2.2: Un esempio di matrice di confusione quando le classi sono solo due [3]

Ogni entry f_{ij} indica il numero di istanze della classe i che si prevede siano di classe j. Per esempio, f_{01} è il numero di istanze della classe 0 predetto erroneamente come classe 1. Il numero di previsioni corrette fatte dal modello è $(f_{11} + f_{00})$ e il numero di previsioni errate è $(f_{10} + f_{01})$.

Anche se una matrice di confusione fornisce le informazioni necessarie per determinare quanto bene funziona un modello di classificazione, riassumere queste informazioni in un unico numero rende più immediato confrontare le prestazioni relative dei diversi modelli. Questo può essere fatto utilizzando delle metriche di valutazione come **Accuracy** e **F1-score**, che vengono calcolate nel seguente modo:

$$\text{Accuracy} = \frac{\text{Number of correct predictions}}{\text{Total Number of predictions}}$$

Per problemi di classificazione binari, la accuracy di un modello (classificatore, DT, ecc.) è data da:

$$\text{Accuracy} = \frac{f_{11} + f_{00}}{f_{11} + f_{10} + f_{01} + f_{00}}$$

Un'altra metrica importantissima, e nei prossimi capitoli i confronti saranno fatti tenendo conto anche di questa metrica, è il già menzionato F1-score che per essere definito nel modo corretto è necessario, prima, introdurre altre due metriche che lo vanno a comporre: la *precision* e il *recall*. La Precision è definita nel modo seguente:

$$\text{precision} = \frac{\text{Number of True Positives}}{\text{Number of True Positives + Number of False Positives}}$$

La Recall è definita invece cosi:

$$\text{recall} = \frac{\text{Number of True Positives}}{\text{Number of True Positives + Number of False Negatives}}$$

Ora che abbiamo definito le due componenti fondamentali del F1-score, calcoliamolo nel seguente modo:

$$\text{F1-score} = 2 * \frac{\text{Precision * Recall}}{\text{Precision + Recall}}$$

Un'altra metrica interessante è l'*Error-rate*, ovviamente più è basso meglio è:

$$\text{Error-rate} = \frac{\text{Number of Wrong Predictions}}{\text{Total Number of Predictions}}$$

Gli algoritmi di apprendimento, per la maggior parte delle tecniche di classificazione, sono progettati per apprendere modelli che raggiungono valori molto alti di accuracy e F1-score e, ovviamente, valori molto bassi di Error-rate quando applicati al testing set.

Ora che si sono chiariti concetti come deduzione, induzione, classificazione, metriche di performance, si dovrebbe finalmente essere pronti a definire la natura e il funzionamento di un Decision Tree Classifier.

2.2 Decision Tree Classifier

Per illustrare come funziona un albero decisionale, consideriamo il problema di voler classificare giornate idonee per giocare a tennis da quelle non idonee utilizzando il dataset di Figura 2.3 a pagina 11.

Day	Outlook	Temperature	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Figura 2.3: PlayTennis dataset [2]

Data una generica giornata, in che modo si può predire se trattasi di una di quelle giornate idonee o meno? Un primo approccio può essere quello di porsi delle domande sugli attributi del dataset (cioè i nomi delle colonne) e quindi, per esempio, ci si potrebbe chiedere se la temperatura sarà mite (mild) o troppo calda (hot); infatti se la temperatura è troppo calda, e magari c'è pure troppo vento (ora si considera anche l'attributo Wind), sarebbe meglio evitare di giocare a tennis quel giorno.

Si capisce quindi che ***il processo di classificazione parte da una serie di domande sui valori degli attributi***: prima si è domandato se fosse una giornata calda oppure no, poi si è domandato se il vento fosse forte o debole, insomma si è esaminato prima un attributo con uno dei suoi possibili valori (*Attributo: Temperature, Valore: Mild*) e poi si è esaminato un altro attributo (*Attributo: Wind, Valore: Strong*) sempre con uno dei suoi possibili valori.

L'esempio precedente illustra come è possibile risolvere un problema di classificazione ponendo una serie di domande attentamente elaborate sugli attributi. Ogni volta che si riceve una risposta, si può subito porre un'altra domanda e cosi via, fino a quando non si riesce a

decidere definitivamente la sua etichetta di classe (giocare a tennis "SI" in quel dato giorno oppure "NO).

La serie di domande e le loro possibili risposte possono essere organizzate in una struttura gerarchica chiamata albero decisionale. La Figura 2.4 a pagina 13 mostra un esempio dell'albero decisionale completo e ultimato per il problema PlayTennis. Un tipico albero decisionale è costituito da tre tipi di nodi:

- **Un nodo radice**, senza collegamenti in entrata e zero o più collegamenti in uscita.
- **Nodi interni**, ognuno dei quali ha esattamente un link in entrata e due o più link in uscita.
- **Nodi foglia** o terminali, ognuno dei quali ha esattamente un collegamento in entrata e nessun collegamento in uscita.

Ogni nodo foglia nell'albero decisionale è associato a un'etichetta di classe, nel caso di PlayTennis le etichette di classe sono solo due: "SI" oppure "NO". PlayTennis è quindi un problema di classificazione con due classi anche detto binario. I nodi non terminali, che includono i nodi radice e interni, contengono condizioni di test degli attributi che vengono in genere definite utilizzando un singolo attributo. Ogni possibile risultato della condizione di test dell'attributo è associato esattamente a un figlio di questo nodo. Ad esempio, il nodo radice dell'albero mostrato nella Figura 2.4 utilizza l'attributo Outlook (previsioni meteo) per definire una condizione di test dell'attributo che ha tre esiti: sunny, overcast e rain risultando in tre nodi figli.

Dato un albero decisionale, classificare un'istanza è semplice: partendo dal nodo radice si applica la condizione di test e, in base al risultato della condizione di test, si segue il ramo propizio che condurrà o ad un altro nodo interno, per il quale viene applicata una nuova condizione di test degli attributi, o ad un nodo foglia. Una volta raggiunto un nodo foglia, tale nodo conterrà l'etichetta da assegnare a quella determinata istanza di test.

Ad esempio, guardando ancora l'immagine 2.4 a pagina 13, si nota che se le previsioni meteo stabiliscono pioggia e il vento è pure forte si arriverà ad una foglia contenente la classificazione "NO", mentre se le previsioni meteo danno sole, con umidità normale, allora la foglia di classificazione a cui si arriva darà un "SI", ma adesso si presti attenzione: se le previsioni me-

teo, che è l'attributo scelto alla radice, da direttamente come valore overcast, arriviamo subito ad una foglia di classificazione alla quale è associata una etichetta di classe "SI".

Quindi da questo esempio il lettore attento capirà che se, con opportuni criteri, *scegliamo già dalla radice i migliori attributi su cui porci domande, l'albero sarà più piccolo e si arriverà più facilmente ad una classificazione.*

Si è spiegato in termini elementari cos'è il bias induttivo che permette all'albero di *imparare* dagli esempi ed essere in grado di predire altri esempi che non ha mai visto (unseen).

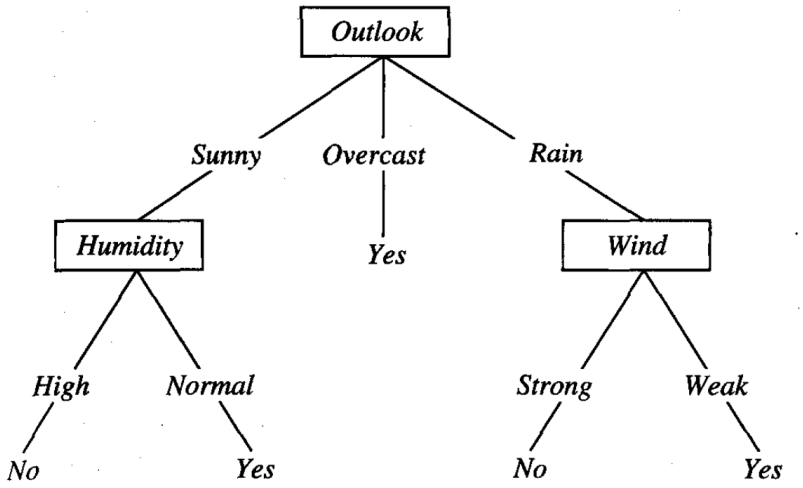


Figura 2.4: Albero di decisione per il dataset PlayTennis [2]

La definizione data da Mitchell sul bias induttivo è la seguente: **"Il bias induttivo di un algoritmo è l'insieme di assunzioni che il classificatore usa per predire l'output dati gli input che esso non ha ancora incontrato".** *Per un decision tree il bias induttivo* utilizzato dall'algoritmo che lo genera (l'algoritmo lo vedremo dopo) *è il preferire alberi* ***meno profondi*** *a quelli più profondi.*

Affinché l'algoritmo di creazione dell'albero generi quello meno profondo tra tutti quelli possibilmente generabili, è necessario selezionare, già a partire dalla radice, solo i migliori attributi, cioè tutti quegli attributi che godono di determinate caratteristiche.

Alberi che posizionano gli attributi aventi *il maggior Gain in impurità statistica* il più vicino possibile alla radice sono da preferire agli altri. Ecco perché nella Figura 2.4 alla radice dell'albero abbiamo come attributo *"Outlook"*.

2.3 L'algoritmo di Hunt

```
TreeGrowth (E, F)
 1: if stopping_cond(E,F) = true then
 2:    leaf = createNode().
 3:    leaf.label = Classify(E).
 4:    return leaf.
 5: else
 6:    root = createNode().
 7:    root.test_cond = find_best_split(E, F).
 8:    let V = {v|v is a possible outcome of root.test_cond }.
 9:    for each v ∈ V do
10:       E_v = {e | root.test_cond(e) = v and e ∈ E}.
11:       child = TreeGrowth(E_v, F).
12:       add child as descendent of root and label the edge (root → child) as v.
13:    end for
14: end if
15: return root.
```

Figura 2.5: Algoritmo di Hunt per un Decision Tree [3]

L'algoritmo di Hunt in Figura 2.5 a pagina 14 utilizza una strategia *greedy* per consentire la crescita *top-down* dell'albero di decisione effettuando una serie di scelte localmente ottime circa quali attributi utilizzare per partizionare i dati di training. E' anche uno degli algoritmi più utilizzati, rappresentando la base per molte implementazioni reali dei DT.

Il MixedDecisionTree utilizza l'algoritmo ID3 (Iterative Dichotomiser 3), ma ci sono anche altre implementazioni reali dell'algoritmo di Hunt come C4.5 e CART che non menzioneremo ulteriormente in questa trattazione.

L'algoritmo di Hunt in Figura 2.5 a pagina 14 presenta uno pseudocodice per l'algoritmo di induzione dell'albero decisionale. L'input di questo algoritmo prevede due parametri, il primo è un insieme di esempi di training E, il secondo rappresenta l'insieme di tutti gli attributi F. L'algoritmo funziona *selezionando ricorsivamente l'attributo migliore per dividere i dati* (come si evince dalla riga 7 di pseudocodice) ed espandendo i nodi dell'albero (righe 11 e 12) fino a quando un *criterio di arresto* non viene soddisfatto (riga 1). I criteri di arresto sono i seguenti:

- Se tutti gli esempi E sono positivi, allora fermati, crea un nodo foglia da restituire e assegnagli la classificazione +

- Se tutti gli esempi E sono negativi, allora fermati, crea un nodo foglia da restituire e assegnagli la classificazione -
- Se l'insieme degli attributi F diventa vuoto, allora fermati, crea un nodo foglia da restituire e assegnali come classificazione il valore di etichetta più ricorrente in E

I dettagli di questo algoritmo sono spiegati di seguito:

1. La funzione **createNode()**, riga 6, estende l'albero delle decisioni creando un nuovo nodo. Un nodo radice o nodo interno nell'albero decisionale viene implementato inserendo al suo interno una condizione di test o condizione di split, ad es. node.test_cond, mentre una foglia avrà al suo interno una etichetta di classe, ad es. node.label
2. La funzione **find_best_split(E,F)**, riga 7, determina la condizione di test degli attributi per il partizionamento delle istanze di addestramento associate a un nodo. L'attributo di divisione scelto dipende da un determinato criterio. Di solito si usa l'impurità statistica. Le misure popolari includono l'entropia e il Gini index, che vedremo nelle prossime sezioni.
3. La funzione **Classify(E)**, riga 3, determina l'etichetta di classe da assegnare a un nodo foglia.
4. La funzione di arresto **stopping-cond(E,F)**, riga 1, viene utilizzata per terminare il processo di crescita dell'albero. Poiché gli alberi decisionali impiegano un approccio di partizionamento ricorsivo top-down per costruire un modello, il numero di istanze di addestramento associate a un nodo diminuisce all'aumentare della profondità dell'albero. Di conseguenza, un nodo foglia può contenere poche istanze di addestramento per prendere una decisione statisticamente significativa sulla sua etichetta di classe. Questo è noto come il problema della frammentazione dei dati. Un modo per evitare questo problema è quello di non consentire la divisione di un nodo quando il numero di istanze associate al nodo scende al di sotto di una certa soglia. Detto ciò, si ritiene opportuno presentare gli iper-parametri di pre-pruning contemplati in questa tesi (si trovano anche nella libreria di sklearn), che consentono di fermare la creazione dell'albero sotto determinate condizioni per evitare una eccessiva specializzazione: **max_depth**, **min_sample_split**

e **min_sample_leaf**. Tutti e tre rappresentano dei valori interi di soglia, in particolare il primo rappresenta un limite superiore che impedisce di creare un nuovo nodo se la profondità corrente dell'albero è strettamente maggiore del valore soglia. Il secondo fa in modo che un nodo non si possa più splittare, diventando una foglia (con classificazione: etichetta che ricorre maggiormente nei dati), se il numero dei samples associati a tale nodo sono strettamente minori del valore intero di soglia. L'ultimo dei tre controlla, prima ancora che il nuovo nodo venga generato, se il potenziale split risulti in un nodo child con meno samples di quelli previsti dall'intero soglia. Se così dovesse accadere allora il nodo diventa una foglia con classificazione: etichetta che ricorre maggiormente nei dati.

Nella prossima sezione si capirà finalmente come si riesce a scegliere gli attributi migliori da collocare il più vicino possibile alla radice affinché sia soddisfatto il bias induttivo previsto dall'algoritmo di Hunt, cioè la *preferenza assoluta nei confronti di alberi meno profondi a quelli più profondi.* In poche parole si arriverà a comprendere che la funzione vista alla riga 7 di pseudo-codice in Figura 2.5, **find_best_split(E,F)** riesce a *selezionare il miglior attributo da splittare basandosi sul concetto di impurità statistica.*

Queste misure cercano di dare la preferenza agli attributi che partizionano le istanze di training set in sottoinsiemi più puri nei nodi figli, la maggior parte dei quali hanno le stesse etichette di classe. Avere nodi più puri è utile poiché un nodo che ha tutte le sue istanze di training della stessa classe non ha bisogno di essere ulteriormente splittato. Al contrario, è probabile che un nodo impuro, cioè contenente istanze di training con classi di valori per la maggior parte tutti diversi richieda diversi livelli di espansioni del nodo, aumentando così notevolmente la profondità dell'albero le cui prestazioni saranno notevolmente degradate.

Gli alberi più grandi sono meno desiderabili in quanto sono più suscettibili all'overfitting, una condizione che può degradare la capacità dell'albero di generalizzare (cioè effettuare predizioni corrette su dati mai riscontrati, unseen, e che vogliamo classificare). Sono anche difficili da interpretare e incorrono in più tempo di formazione e test rispetto agli alberi più piccoli che sono quelli più desiderabili.

Verranno presentati diversi modi di misurare *l'impurità di un nodo e l'impurità collettiva dei suoi nodi figli*, entrambi utilizzati per individuare il **miglior attributo da splittare**.

2.4 Misure di Impurità

Si riesce a trovare il best split attribute attraverso misure di impurità note come Entropia, Gini Index e Misclassification Error (anche se quest'ultima è piuttosto didattica ergo non utilizzata nelle implementazioni reali). La trattazione verrà impostata presentando prima le misure di impurità per un nodo singolo e poi quelle collettive, cioè che coinvolgono anche i nodi figli.

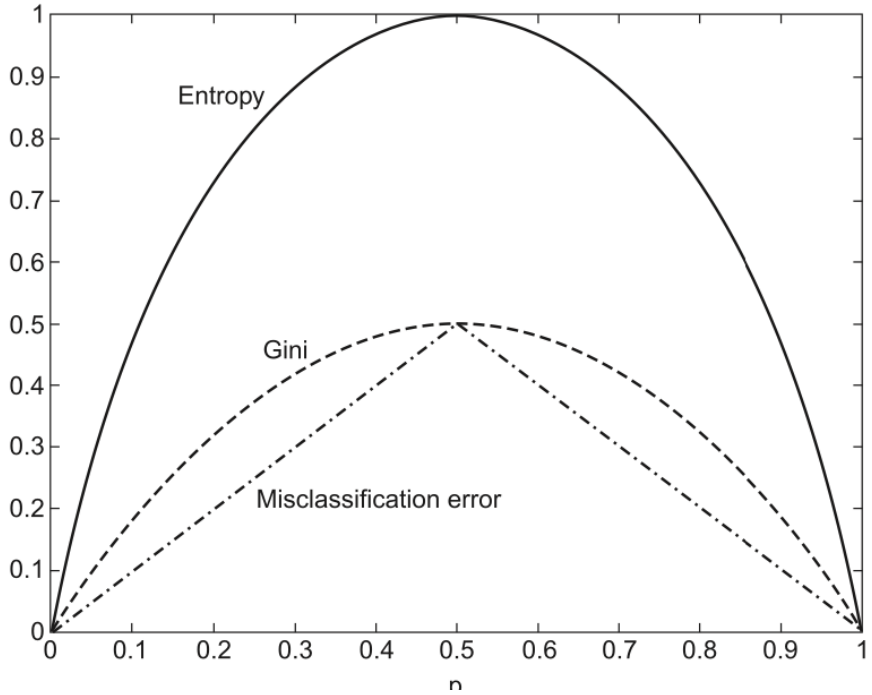


Figura 2.6: Confronto tra le misure di impurità per classificazioni binarie [3]

2.4.1 Misure di Impurità per un nodo singolo

L'impurità di un nodo misura quanto sono dissimili le etichette di classe per le istanze di dati appartenenti ad un nodo comune. Di seguito sono riportati esempi di misure che possono essere utilizzate per valutare l'impurità di un nodo t:

$$
\begin{aligned}
\text{Entropy} &= -\sum_{i=0}^{c-1} p_i(t) \log_2 p_i(t), \\
\text{Gini index} &= 1 - \sum_{i=0}^{c-1} p_i(t)^2, \\
\text{Classification error} &= 1 - \max_i [p_i(t)],
\end{aligned}
$$

Figura 2.7: Confronto tra le misure di impurità per classificazioni binarie [3]

Dove $p_i(t)$ è la frequenza relativa delle istanze di addestramento al nodo t che appartengono alla classe i, c è il numero totale di classi e $0log2(0)$ lo poniamo uguale a zero quando appare nei calcoli dell'entropia. Tutte e tre le misure danno un valore di impurità zero se un nodo contiene solo istanze appartenenti ad una singola classe, mentre danno il massimo valore dell'impurità, se il nodo ha una proporzione uguale di istanze di più classi. La Figura 2.6 a pagina 17 confronta l'entità relativa delle misure di impurità quando applicate ai problemi di classificazione binaria. Poiché ci sono solo due classi, $p_0(t) + p_1(t) = 1$.

L'asse orizzontale p si riferisce alla frazione di istanze che appartengono a una delle due classi. Ciò che è importante da osservare è che *tutte e tre le misure raggiungono il loro massimo quando la distribuzione della classe è uniforme* (cioè $p_0(t) = p_1(t) = 0.5$) e *il valore minimo, cioè* 0*, quando tutte le istanze appartengono a una singola classe* (cioè $p_0(t)$ o $p_1(t)$ è uguale a 1).

Per rendere tutto più chiaro prestiamo attenzione al seguente esempio:

Node N_1	Count
Class=0	0
Class=1	6

$\text{Gini} = 1 - (0/6)^2 - (6/6)^2 = 0$
$\text{Entropy} = -(0/6)\log_2(0/6) - (6/6)\log_2(6/6) = 0$
$\text{Error} = 1 - \max[0/6, 6/6] = 0$

Node N_2	Count
Class=0	1
Class=1	5

$\text{Gini} = 1 - (1/6)^2 - (5/6)^2 = 0.278$
$\text{Entropy} = -(1/6)\log_2(1/6) - (5/6)\log_2(5/6) = 0.650$
$\text{Error} = 1 - \max[1/6, 5/6] = 0.167$

Node N_3	Count
Class=0	3
Class=1	3

$\text{Gini} = 1 - (3/6)^2 - (3/6)^2 = 0.5$
$\text{Entropy} = -(3/6)\log_2(3/6) - (3/6)\log_2(3/6) = 1$
$\text{Error} = 1 - \max[3/6, 3/6] = 0.5$

E' bene notare che se si hanno delle istanze tutte appartenenti ad una stessa classe in un nodo, il valore dell'entropia, del Gini Index e del Misclassification Error sarà pari al loro valore più basso, cioè 0. Lo zero rappresenta il valore più basso dal momento che non esistono valori negativi di impurità statistica.

Se invece in un nodo, ad esempio, su sei istanze totali con due sole classi di classificazione, se ne hanno tre etichettate come "SI" e altre tre etichettate come "NO", allora in corrispondenza di quel nodo si avrà il massimo grado di impurità, cioè l'Entropia sarà pari al suo valore massimo 1, mentre il Gini Index e il Misclassification Error saranno entrambi pari al loro valore massimo 0.5.

2.4.2 Impurità post parent split dei nodi figli

Si consideri una condizione di test degli attributi che divide un nodo contenente N istanze di addestramento in k figli, $v_1, v_2, ..., v_k$, dove ogni nodo figlio rappresenta una partizione dei dati derivanti da uno dei k risultati della condizione di test degli attributi. Sia $N(v_j)$ il numero di istanze di addestramento associate a un nodo figlio v_j, il cui valore di impurità è $I(v_j)$. Poiché un'istanza di addestramento nel nodo padre raggiunge il nodo v_j per una frazione di $N(v_j)/N$ volte, *l'impurità collettiva dei nodi figli può essere calcolata attraverso una media pesata delle impurità dei nodi figli*, come segue:

$$I(\text{children}) = \sum_{j=1}^{k} \frac{N(v_j)}{N} I(v_j),$$

A questo punto è possibile finalmente identificare l'attributo migliore da splittare. Per determinare la bontà di una condizione di split, è opportuno confrontare il grado di impurità del nodo padre (pre-split) con il grado ponderato di impurità dei nodi figli (post-split). Più grande è la loro differenza, più appetibile diventa la scelta di quel particolare attributo. Questa differenza, nota anche come Gain, guadagno in purezza, sarà ciò che ci farà propendere per un attributo piuttosto che per un altro.

In soldoni, l'attributo avente il massimo gain verrà selezionato tra tutti gli altri e verrà splittato, soddisfacendo egregiamente il bias induttivo dell'algoritmo ID3 di machine learning.

Si definisce l'impurità dei figli come la media ponderata delle loro impurità; quindi, il guadagno in impurità sarà dato dalla seguente equazione:

Gain = Impurità del padre - Impurità dei figli

2.5 Caratteristiche finali dei Decision Tree

A questo punto della trattazione il lettore dovrebbe essere nella condizione di aver capito, almeno in teoria, come funziona un classificatore di tipo Decision Tree.

L'algoritmo di Hunt ne spiega il funzionamento meglio di mille parole: ad ogni chiamata ricorsiva, se non vi sono condizioni di arresto, viene creato sempre un nuovo nodo di espansione con al suo interno l'attributo di best split ricavato grazie alla scelta golosa fornita dal Gain (guadagno in impurità). L'attributo che in quel momento presenta il massimo guadagno in impurità verrà selezionato a prescindere da quale criterio si sia scelto per l'impurità statistica. La differenza, tra Entropia o indice di Gini, è tutta nella loro formula quando si va ad effettuare il calcolo in un nodo singolo t. Se sono necessari ulteriori chiarimenti si riveda la Figura 2.7 a pagina 18.

L'ultima parte interessante da capire è il perché i DT sono così famosi e utilizzati rispetto ad altre tecniche di classificazione che si hanno a disposizione nel machine learning. Si daranno qui alcune delle seguenti motivazioni:

- Facile da interpretare soprattutto laddove l'albero risultante non sia troppo profondo
- L'accuracy è paragonabile a quella che si otterrebbe da altre tecniche di classificazione per molti semplici datasets
- Robusti ai dati rumorosi: i DT possono facilmente gestire attributi ridondanti (un attributo ridondante si definisce tale quando è fortemente correlato ad un altro attributo nei dati) dal momento che essi mostrano dei gain in impurità molto simili tra loro se sono selezionati per lo split, ma alla fine solo uno di loro verrà selezionato (quello col massimo gain) dall'algoritmo di Hunt come migliore attributo di split. Robusto anche ai dati irrilevanti (un dato si definisce irrilevante quando è molto poco associato alla variabile

target, cioè la colonna nel data set che ha solo etichette di classificazione) dal momento che, avendo uno scarsissimo guadagno in impurità, esiste un bassissimo rischio che possano essere scelti dall'algoritmo come attributo di best split.

- Sono poco costosi in termini di risorse. Costruire un albero costa relativamente poco.
- Una volta che l'albero è stato costruito e trainato bene, esso sarà sempre estremamente veloce nel classificare nuovi dati unseen, infatti *per trainare un albero la complessità è* ***O(M * N * logN)*** dove M è pari al numero di colonne o attributi presenti nel dataset, invece N rappresenta il numero di istanze, oggetti o righe nel dataset. *La complessità invece per la sola classificazione di nuovi record è* ***O(w)*** dove w rappresenta la massima profondità dell'albero.
- Gestiscono bene anche i valori mancanti negli attributi. [1]

Ma quindi i DT hanno solo pregi? In realtà per quanto incredibilmente validi e ottimi, anche i DT non sono esenti da difetti:

- Lo spazio in memoria per alcuni alberi può diventare esponenzialmente grande, il che rende quasi vana la ricerca di trovare il miglior DT, nonostante gli approcci greedy.
- Non prendono in considerazione alcune interazioni che potrebbero esistere tra i vari attributi
- Ogni decisione coinvolge solo un attributo, il che rappresenta un limite.
- Frammentazione dei dati, cioè il numero delle istanze si riduce ad ogni livello di albero attraversato, di conseguenza il numero di istanze che arrivano ai nodi foglia può essere troppo basso impedendoci quindi di prendere una decisione statisticamente rilevante.
- Tree Replication: può succedere che alcuni sottoalberi possano apparire uguali in più rami.

Capitolo 3

Implementazione

Software and cathedrals are much the same: first we build them, then we pray.
(Sam Redwine)

In questo capitolo verrà spiegato il funzionamento interno del *MixedDecisionTree* o MDT, enfatizzando molto la parte implementativa e, a tal proposito, si ritiene opportuno specificare le librerie utilizzate nella realizzazione di questo progetto e la loro versione: **numpy** in versione 1.21.5, **pandas** in versione 1.4.4, **scikit-learn** in versione 1.1.1 e **matplotlib** in versione 3.5.2.

Come già anticipato nell'introduzione, il MDT è in grado di acquisire in input dataset in cui i valori categorici sono espressi mediante stringhe e i valori continui espressi dal tipo float, cioè i numeri reali in virgola mobile (eventuali colonne a valori interi verranno convertite in float).

Sostanzialmente il MDT prende in input gli stessi dataset che vediamo nei libri di teoria, come ad es. PlayTennis nel libro di Machine Learning di Mitchell [2], analizzato nel capitolo precedente a pagina 11 in Figura 2.3 o altri dataset misti in cui compaiono sia stringhe di valori categorici, sia interi che float ecc.

Se si volesse dare in input al decision tree di sklearn il dataset PlayTennis così com'è, senza apportarvi alcuna modifica, questi, solleverebbe istantaneamente una miriade di eccezioni. Tutto ciò accade perché l'albero della libreria scikit-learn acquisisce direttamente solo valori continui, sotto forma di interi o float, e non contempla minimamente gli attributi categorici.

Il MDT, invece, preso in input il dataset PlayTennis, crea all'istante l'albero senza sollevare alcuna eccezione e il decision tree che ne viene fuori sarà esattamente pari a quello visto nella Figura 2.4 di pagina 13. Si presti attenzione al fatto che il nodo radice che è padre di ben 3 figli. Ergo *non sono contemplati solo split binari.*

La domanda a questo punto potrebbe essere: "come se la cava il MDT invece con dataset completamente continui come, per esempio, il famosissimo Iris a 3 classi?" La risposta è altrettanto semplice: acquisisce facilmente in input anche lui, ma dovrà sottoporre il dataframe ad una serie di modifiche che avvengono esternamente all'albero. Con attributi continui, il MDT effettuerà split binari.

Superata la fase di training, il MDT raggiungerà praticamente la stessa accuracy e f1-score rispetto a quello di sklearn. L'unica tangibile differenza riguarda la complessità in tempo e spazio: il MDT è molto meno efficiente rispetto a quello di sklearn e sarà possibile constatarlo nel quarto capitolo osservando la tabella e la sua descrizione a pagina 66.

Tornando al MDT, esso supporta **tre modalità**: la prima è denominata *"classic"* e prevede il funzionamento standard dell'albero come descritto fin qui, la seconda è denominata *"median"* e utilizza la funzione della libreria pandas.cut con mid; sostanzialmente questa seconda modalità funziona come la modalità classica, ma *la semplifica tantissimo* senza perdere troppo in accuracy e f1-score, anzi a volte l'accuracy rimane addirittura la stessa, ma il guadagno computazionale in tempo e spazio è notevole. La terza ed ultima modalità è denominata *"binned"* e utilizza solo la pandas.cut con un numero di bins variabili trasformando colonne di valori continui in stringhe di intervalli, lasciando immutate le colonne con valori categorici.

Ha avuto senso quindi intraprendere questo progetto e programmare da zero un classificatore di tipo decision tree? Seguono alcune motivazioni a favore:

- Facilità di lettura per l'utente: ogni nodo radice e interno supporta il multi-split, ergo avremo la visualizzazione dell'albero esattamente come quella fornitaci dalla letteratura. Si indica nuovamente il DT risultante dal dataset PlayTennis a pagina 13 come esempio visivo.

- I dataset sono più espressivi e anch'essi di più facile lettura: gli attributi categorici sono espressi mediante stringhe e vengono letti direttamente dal MDT.

- Tre modalità di funzionamento: *classic, median* e *binned*, tutte nello stesso pacchetto, rendono il MDT più versatile e adatto ad un maggior numero di confronti.

Nella prossima sezione sarà già possibile immergersi completamente nell'implementazione del MDT partendo dalle classi principali che lo compongono.

3.1 Classe interna Node e Classe MixedDecisionTree

Si è ritenuto opportuno inserire la classe *Node.py* all'interno della classe vera e propria dell'albero *MixedDecisionTree.py* affinché il tutto sia più facilmente importabile in altri programmi.

```
1  class MixedDecisionTree:
2      class Node:
3          def __init__(self):
4              self.split_condition = None
5              self.gain: float = float()
6              self.split_value = None
7              self.isLeaf: bool = False
8              self.isRoot: bool = False
9              self.classification = None
10             self.parent = None
11             self.depth: int = int()
12             self.children = list()
13
14     def __init__(self, class_name='class', max_allowed_depth=50, min_samples_split=2, min_
           samples_leaf=1,
15                  tree_mode='classic', number_of_bins=0, cut_threshold=10000, criterion='
                        entropy'):
16         self.root = None
17         self.class_name = class_name
18         self.initial_depth = 0
19         self.max_allowed_depth = max_allowed_depth
20         self.min_samples_split = min_samples_split
21         self.min_samples_leaf = min_samples_leaf
22         self.tree_mode = tree_mode
23         self.number_of_bins = number_of_bins
24         self.cut_threshold = cut_threshold
25         self.criterion = criterion
26         self.most_common_value = None
27         # ...
```

Listing 3.1: MixedDecisionTree e la classe interna Node

Prestando attenzione alla inner class Node, nel Listato di codice 3.1, alla riga 4 troviamo la condizione di test per il miglior attributo da splittare, come noto dal capitolo precedente, il miglior attributo da splittare viene scelto in base al massimo gain (Gain = Impurità del nodo parent - Impurità dei figli) e può essere calcolato usando come misura di impurità l'Entropia o il Gini index. Alla riga 9 verrà salvato il valore di classificazione se il nodo in questione è una foglia, quindi self.classification = etichetta di classificazione. Le righe 7 e 8 informano l'utente circa la natura dei nodi, se ad es. trattasi di nodo radice o nodo foglia, la riga 11 memorizza la profondità in cui si trova il nodo corrente, e, cosa importantissima, la riga 12 porta all'attenzione che *nel nodo è implementata una lista di figli*, feature comodissima dato che da essa deriva il supporto al **multi-split** dei nodi interni (radice inclusa), cioè la possibilità che ogni nodo possa avere un numero arbitrario di figli.

Nella classe "MixedDecisionTree" troviamo, oltre agli iper-parametri di pre-pruning, anche i parametri di configurazione da assegnare al momento dell'istanziazione della classe:

- **self.criterion** stabilisce se l'albero userà, per trovare il best attribute da splittare, l'entropia o il Gini index
- **self.tree_mode** stabilisce in che modalità l'albero dovrà trovarsi, come detto in precedenze, le modalità sono tre: classic, median, binned
- **self.number_of_bins** non utilizzata in modalità classic, ma solo in modalità binned e median, stabilisce il numero di bins che verranno usati nella funzione pandas.cut
- **self.cut_threshold** solo la modalità classic può farne uso, in sostanza è un intero soglia che serve per ridurre il numero di righe in un dataset. Se ho 7000 righe di dataset e la soglia e fissata a 3000, il mio dataframe risultante conterrà solo le prime 3000 righe, le restanti 4000 mila righe non verranno considerate in quanto non presenti nel dataframe.
- **self.max_allowed_depth** iper-parametro di pre-pruning, serve a far stoppare la crescita dell'albero una volta superata la profondità massima prestabilita.
- **self.min_samples_split** parametro di pre-pruning, serve ad impedire ad un nodo di splittarsi se i suoi samples sono strettamente minori al valore del parametro. Se co-

sì dovesse accadere il nodo non si splitta, bensì diventa una foglia con classificazione: etichetta più ricorrente nei dati.

- **self.min_samples_leaf** parametro di pre-pruning che controlla, ancor prima che il nuovo nodo venga generato se il potenziale split risulti in un nodo child con meno samples di quelli previsti dall'intero soglia. Se così dovesse accadere allora il nodo diventa una foglia con classificazione: etichetta che ricorre maggiormente nei dati.

3.2 Impurità e calcolo del Gain

E' arrivato il momento di mostrare il metodo che si occupa di calcolare il Gain al fine di selezionare il best split attribute. Il calcolo viene effettuato usando l'entropia o il Gini index in base alla scelta dell'utente al momento dell'instanziazione del MDT. La scelta del criterio viene memorizzata all'interno della variabile di istanza self.criterion.

```
1  def _get_impurity(self, attribute):
2      values, counts = np.unique(attribute, return_counts=True)
3      impurity_values = []
4      if self.criterion == 'entropy':
5          for i in range(len(values)):
6              impurity_values.append(-(counts[i] / np.sum(counts)) * (np.log2(counts[i] /
                   np.sum(counts))))
7          impurity_measure = np.sum(impurity_values)
8      elif self.criterion == 'gini':
9          for i in range(len(values)):
10             impurity_values.append((counts[i] / np.sum(counts)) ** 2)
11         impurity_measure = 1.0 - np.sum(impurity_values)
12     else:
13         raise Exception("Wrong criterion, possible choices are: 'entropy' or 'gini'")
14     return impurity_measure
15
16 def _getGain(self, data, split_attribute_name):
17     I_parent = self._get_impurity(data[self.class_name])
18     assert (I_parent >= 0)
19     values, counts = np.unique(data[split_attribute_name], return_counts=True)
20     impurity_values = []
21     for i in range(len(values)):
22         impurity_values.append(
23             (counts[i] / np.sum(counts)) * self._get_impurity(
```

```
24              data.where(data[split_attribute_name] == values[i]).dropna()[self.class_
                    name]))
25      I_child = np.sum(impurity_values)
26      gain = I_parent - I_child
27      return gain
```

Listing 3.2: calcolo del Gain

Alla riga di codice 16 del Listato 3.2, troviamo il medoto ausiliario _get_Gain il cui scopo è quello di effettuare il calcolo dell'impurità di un singolo attributo passato in input. Se il criterio scelto al momento in cui è stato istanziato l'albero è 'entropy', il calcolo verrà effettuato tenendo conto dell'Entropia: le righe di codice dalla 5 alla 7 effettueranno il calcolo dell'entropia, mentre se il criterio scelto è 'gini' allora il calcolo effettuato terrà conto del Gini index e sarà esattamente pari a quello calcolato alle righe di codice da 9 a 11.

Il metodo _getGain estenderà il calcolo a tutti i potenziali figli, fino a trovare il valore del gain per quell'attributo, come possibile notare alla riga di codice 26 del Listato 3.2:
$Gain = I_{parent} - I_{child}$ e lo restituirà alla funzione incaricata di scegliere il best attribute. Il calcolo di I_{child} avviene alle righe da 21 a 25 e si può considerare come la media pesata dell'impurità dei figli post-split.

3.3 Il Nucleo di tutto: L'algoritmo ID3

Finalmente è giunto il momento di arrivare dritti al nucleo del MixedDecisionTree, cioè la funzione ricorsiva che costruisce l'albero e da cui partono praticamente tutte le chiamate agli altri metodi. Trattasi del metodo che implementa l'algoritmo ricorsivo ID3 (Iterative Dichotomiser 3). Per comprendere più facilmente le righe che seguono, si consiglia di leggere lo pseudo-codice dell'algoritmo di Hunt a pagina 14.

La prima cosa a cui si va incontro sono i criteri di stop, quindi si controllano le condizioni di arresto: se esse si verificano, allora l'algoritmo si ferma e viene generata una foglia avente classificazione idonea a quel particolare caso, dopodiché viene restituita.

Nella versione implementata in questa tesi, oltre ai criteri di stop, si sono aggiunte anche le condizioni di pre-pruning che hanno sempre il medesimo fine di arrestare prematuramente la crescita dell'albero.

Dalla teoria sappiamo che gli alberi decisionali migliori sono, a partire dagli stessi dati di input, la loro versione meno profonda possibile (bias induttivo).

Se i criteri di arresto o quelli di pre-pruning non si verificano, allora l'albero inizia e continua la sua espansione istanziando, ad ogni chiamata ricorsiva, un nuovo nodo a cui viene assegnato il best split attribute. *Per ogni valore unico dell'attributo di split facciamo crescere un ramo. La ricorsione verrà effettuata su un insieme di samples sempre più ristretto rispetto a quello iniziale*, e andrà avanti finché o un criterio di stop o uno di pre-pruning porrà fine alla crescita dell'albero.

3.3.1 Criteri Di Stop

Lo snippet che segue mostrerà i criteri di stop dell'albero, così come previsti dall'algoritmo di Hunt, la versione applicata ID3 mantiene gli stessi.

```
1   def _stoppingCriteria(self, data, attributes):
2       node = MixedDecisionTree.Node()
3       node.isLeaf = True
4       # primo criterio di stop
5       if len(np.unique(data[self.class_name])) <= 1:
6           node.classification = np.unique(data[self.class_name])[0]
7       # secondo criterio di stop
8       elif len(data) <= 0:
9           node.classification = self.most_common_value
10      # terzo criterio di stop
11      elif len(attributes) <= 0:
12          node.classification = self._getMostCommonValue(data)
13      else:
14          # Se nessun criterio di stop o pruning si verifica, restituisco
15          return None, False
16      return node, True
```

Listing 3.3: criteri di stop del MixedDecisionTree

La comprensione di tali criteri appare molto semplice, infatti, per il **primo criterio di stop** (riga 5 dello snippet 3.3) l'albero si arresta se tutti i samples nel nodo hanno la stessa classificazione.

Il **secondo criterio di stop** (riga 8) invece stabilisce che se il numero dei sample all'interno del nodo è pari a zero, allora restituisco una foglia con classificazione: "valore di etichetta

più comune presente nel dataset originale", cioè il valore di moda statistica della colonna di classificazione del dataframe originale; per originale si intende il dataframe intero, prima che iniziasse a restringersi per via delle chiamate ricorsive.

Alla riga di codice 11 appare il **terzo criterio di stop**, il quale afferma che se la lunghezza della lista di attributi dovesse annullarsi, detto in altre parole, se la lista di attributi in gioco diventa vuota, allora restituisco una foglia con classificazione: valore di etichetta più frequente, non nel dataset originale, come accadeva prima, ma il valore più ricorrente nei dati attuali, cioè quelli che si vanno via via restringendo a colpi di chiamate ricorsive.

3.3.2 Criteri di Pre-Pruning

Per completare l'argomento "criteri di arresto", si mostrano le condizioni di pre-pruning dell'albero che hanno lo stesso effetto dei criteri di arresto: fermare prematuramente la crescita dell'albero.

```
1   # PRUNING: MAX_DEPTH
2   if current_depth > self.max_allowed_depth:
3      node = MixedDecisionTree.Node()
4      node.isLeaf = True
5      node.classification = self._getMostCommonValue(data)
6      return node
7
8   # PRUNING: MIN_SAMPLES_SPLIT
9   if len(data) < self.min_samples_split:
10     node = MixedDecisionTree.Node()
11     node.isLeaf = True
12     node.classification = self._getMostCommonValue(data)
13     if current_depth == self.initial_depth:
14     return node
15
16   sub_data_pairs_list = list()
17     flag_leaf = False
18     # per ogni valore dell'attributo di split facciamo crescere un ramo:
19     for value in np.unique(data[best_split_attribute]):
20        split_attribute_value = value
21        # considero il sottinsieme del dataset su cui inizia la ricorsione
22        sub_data = data.where(data[best_split_attribute] == split_attribute_value).
              dropna()
23        # PRUNING: MIN_SAMPLE_LEAF
24        # I sample dei sottodati devono essere strettamente minori dell'iperparametro
```

```
25          if len(sub_data) < self.min_samples_leaf:
26              flag_leaf = True
27              break
28          sub_data_pairs_list.append((split_attribute_value, sub_data))
29
30      if flag_leaf:
31          # codice per fare foglia
32          node = MixedDecisionTree.Node()
33          node.isLeaf = True
34          node.classification = self._getMostCommonValue(data)
35          if current_depth == self.initial_depth:
36              node.isRoot = True
37          return node
```

Listing 3.4: criteri di pre-pruning del MixedDecisionTree

I primi due criteri di pre-potatura dell'albero sono molto immediati e semplici da capire, alla riga 2 del Listato di codice 3.4 si osserva una semplice condizione: se la profondità corrente dell'albero supera la soglia descritta dall'intero *max_depth*, subito ci si ferma restituendo una foglia con classificazione: "etichetta più frequente nei dati fino a quel punto".

Il discorso resta ancora molto semplice se pensiamo al *min_sample_split*, la condizione imposta alla riga 10, afferma che se il numero dei campioni nel nodo è strettamente minore dell'intero soglia, allora quel nodo non verrà più splittato, diventando una foglia con etichetta di classificazione: "valore di classe più frequente nei dati correnti".

Un po' meno intuitivo da comprendere inizialmente è il *min_sample_leaf* (la condizione si trova alla riga 26 del Listato 3.4): trattasi di un intero soglia che controlla, prima ancora di splittare il nodo corrente, che quando esso verrà splittato, se tra uno dei figli ne esiste anche solo uno per cui il numero dei campioni sarebbe inferiore al valore soglia, allora stop, non si splitta più il nodo corrente, piuttosto si va a restituire una foglia con classificazione: "valore di etichetta più ricorrente nei dati attuali".

3.4 Gestione degli Attributi Continui e Categorici

In questa sezione sarà possibile capire la gestione degli attributi continui e categorici in varie parti dell'albero. La prima parte che verrà analizzata sarà quella nell'algoritmo ID3, implemen-

tata nel metodo ricorsivo "_fit", la seconda parte vedrà protagonista un altro metodo ricorsivo "predict", il cui scopo è quello di capire quanto bene (o male) generalizza il MDT.

Si ricorda al lettore che il termine "generalizza" bene significa quanto bene l'albero decisionale riesce a predire istanze nuove, cioè mai riscontrate prima, ed etichettarle nel modo corretto mantenendo, quindi, una buona accuracy e f1-score.

Si effettuerà una gestione distinta degli attributi continui e categorici anche nel metodo ricorsivo displayMDT, la cui funzione è quella di stampare in formato testo il MDT livello per livello. L'implementazione varrà data nella sezione 3.6 a pagina 41.

3.4.1 Gestione attributi nel metodo fit

```
1  # ...
2  for pair in sub_data_pairs_list:
3  split_attribute_value = pair[0]
4  sub_data = pair[1]
5
6  # partono le chiamate ricorsive che fanno crescere il MDT in profondita'
7  child = self._fit(sub_data, remaining_features, current_depth + 1)
8
9  child.split_value = split_attribute_value
10 child.parent = root
11 child.depth = current_depth + 1
12 if is_numeric_dtype(split_attribute_value):
13    # la funzione is_numeric_type_ considera il valore '?'
14    # come un attributo numerico e non come carattere o stringa
15    if split_attribute_value != '?':
16       if split_attribute_value == 1.0:
17          root.children.insert(0, child) # inserisco come primo figlio o figlio di
                posizione 0 left
18       elif split_attribute_value == 0.0:
19          root.children.insert(1, child) # inserisco come secondo figlio o figlio di
                posizione 1 right
20       else:
21          raise Exception("fatal error... arrivato un split attribute value non
                numerico...")
22    else:
23       raise Exception("Ho trovato un valore ? che va sostituito con un valore valido
             nel file .csv")
24 else: # se invece l'attributo e' categorico, ergo object in python
25    root.children.append(child)
```

Listing 3.5: Gestione attributi continui e categorici nel metodo "fit"

Dal Listato di codice 3.5 si evince che il MixedDecisionTree è stato programmato per gestire naturalmente gli attributi categorici (opposto a quanto fatto da sklearn, che gestisce in modo naturale gli attributi continui), infatti in una sola riga di codice si è gestito il child se il valore dell'attributo scelto come best attribute da splittare è categorico.

La riga di codice 25 semplicemente inserisce il nodo child nella lista dei figli del nodo root, che ora diventa il parent (del nodo child) precedentemente creato per l'espansione. Ciò che desta stupore è che se il nodo root, quindi ora il parent, ha un attributo con 10 valori categorici unici, il suo potenziale split consiste di 10 figli, tutto ciò al fine di enfatizzare ancora una volta le capacità multi-split del MDT che supporta un numero indefinito di figli.

Dalle righe di codice 12 a 23 dello stesso Listato, invece, si gestisce il valore di un attributo continuo.

3.4.2 Gestione attributi nel metodo predict

Si è visto come si sono gestiti gli attributi continui e categorici nel metodo _fit, ma anche il metodo predict deve gestire separatamente i due tipi di attributi per funzionare correttamente. Scopo del metodo predict è quello di, preso in input un esempio unseen, restituire una etichetta di classificazione per quell'esempio mai riscontrato prima.

Se il training dell'albero è stato effettuato bene, allora esso generalizzerà altrettanto bene: effettuerà quindi previsioni corrette mantenendo un ottimo livello di accuracy e f1-score.

```
1   def predict(self, X_test, y_test):
2       X_test[self.class_name] = y_test
3       examples = X_test.iloc[:, :-1].to_dict(orient="records")
4       predicted_list = list()
5       for i in range(len(X_test)):
6           predicted_list.append(self._predict(self.root, examples[i]))
7       return predicted_list
8   def _predict(self, root, example):
9       if root is None:
10          raise Exception("istanzia l'albero e facci il training prima di invocare il
                metodo predict")
11      if root.isLeaf:
```

```
12          return root.classification
13      if ' <= ' in root.split_condition:
14          temp_string = root.split_condition
15          # se ho una split condition continua alla radice:
16          whole_condition = re.split(' <= ', temp_string)
17          condition_name: str = str(whole_condition[0])
18          condition_number: float = float(whole_condition[1])
19          if len(root.children) > 1:
20              if float(example[condition_name]) <= condition_number:
21                  return self._predict(root.children[0], example)
22              else:
23                  return self._predict(root.children[1], example)
24          return self.most_common_value
25      else:
26          # se la split condition e' categorica
27          example_value = example[root.split_condition]
28          for child in root.children:
29              if child.split_value == example_value:
30                  return self._predict(child, example)
31          return self.most_common_value
```

Listing 3.6: Gestione attributi continui e categorici nel metodo predict

Dopo che si è effettuato il training dell'albero si vuole capire come esso generalizzi; quindi, lo step successivo è quello di invocare il metodo *predict* passandogli in input degli esempi unseen da classificare.

Alla riga 4 del Listato 3.6, viene creata una lista che conterrà le predizioni effettuate dal MDT (se si volesse testare l'accuracy e l'f1-score, si dovrebbe disporre anche di un testing set che per definizione possiede già le etichette con le "vere" classificazioni, in modo da confrontarle con le predizioni del nostro albero, è ovvio che se il MDT avrà predetto correttamente ogni esempio passatogli, avremo il 100% di accuracy) con un ciclo for, alla riga 5, passo ciascun esempio alla funzione ricorsiva _predict, la quale restituirà una predizione che si andrà ad inserire nella lista contenente le predizioni.

Definita alla riga di codice 8 del Listato 3.6, cerchiamo di capire come funziona la _predict internamente. Come più volte detto, il MDT gestisce gli attributi categorici direttamente e con naturalezza, poiché è stato progettato proprio con questo scopo in mente, invece, gli attributi continui non vengono gestiti direttamente e bisogna quindi renderli compatibili, cioè si deve sottoporre il dataframe alla trasformazione accennata nella sezione 3.4.1 a pagina 34.

Una volta resi compatibili gli attributi a valori continui, questi saranno trattati come normalissimi attributi categorici in cui i possibili valori possono essere solo 0 e 1, a livello implementativo essi verranno trattati come attributi categorici in cui le liste di children incapsulate nella radice e nei nodi interni avranno solo due figli, il figlio sinistro in posizione 0 della lista e quello destro in posizione 1 della lista.

Per riconoscere gli attributi continui, basta notare che essi avranno tutti la struttura: "nome-colonna-originale <= valore-unico-continuo", di conseguenza è sufficiente scrivere una condizione come quella alla riga 13 del listato 3.6.

Grazie a quella condizione si sa che il nodo root in questione presenta un attributo continuo che, mediante trasformazioni di stringhe (righe da 14 a 18), è possibile confrontarlo con quello degli esempi unseen di input: se la condizione è vera, la funzione _predict chiamerà se stessa con parametri più specifici: al posto di root, si avrà root.children[0], quindi si fa partire la ricorsione sul figlio sinistro, altrimenti, se la risposta è negativa, la si fa partire sul figlio destro, cioè root.children[1].

Al termine della ricorsione si arriverà inevitabilmente ad un nodo foglia (che sono quelli che contengono la classificazione) e, se la predizione sarà corretta, restituiremo la classificazione giusta altrimenti l'albero avrà effettuato un miss. Per capire a quanto ammonta l'accuracy, o l'f1-score, basta confrontare la lista contenente tutte le predizioni con quella contenente le predizioni esatte.

Una volta effettuato il test e riscontrato che l'albero presenta un'ottima accuracy e f1-score (quindi generalizza bene) verrà finalmente utilizzato per far ciò per cui è stato realmente concepito: **classificare**, cioè assegnare un'etichetta a esempi unseen per cui **non** abbiamo già una etichetta.

Finalmente l'albero inizierà a classificare istanze e nel fare ciò sarà velocissimo, infatti una volta trainato l'albero, testato e constatato che tutto procede per il verso giusto, utilizzarlo per classificare esempi mai riscontrati prima (unseen) sarà un'operazione velocissima ed efficientissima; la complessità in tempo, come già sappiamo, sarà $O(w)$, dove w rappresenta la massima profondità.

Dato che, e ormai questa cosa dovrebbe essere scolpita nella pietra, il bias induttivo, grazie al quale un albero di decisione *apprende*, è proprio quello di mantenere l'albero il meno

profondo possibile; quindi, *la w sarà la minima w per quell'albero.*

3.5 Trasformazione degli attributi continui in categorici

Come il MDT modifica il dataframe per gestire gli attributi continui?

Si supponga di avere a disposizione il dataset **Iris**, tale dataset consiste di 5 colonne: "Sepal-length", "Petal-length", "Sepal-width", "Petal-width", "class". La colonna class rappresenta la variabile dipendente Y contenente tutte le etichette di classificazione, ergo non verrà considerata per la trasformazione che segue.

Si consideri la prima colonna, "Sepal-length", e, per semplificare, si assegnino a questa solo 5 valori continui come ad es. 5.6, 4.8, 5.6, 6.4, 4.8; *il primo step da fare è l'ordinamento dei valori dal più piccolo al più grande, il secondo quello di eliminare, se esistono, tutti i duplicati*, lasciando quindi, nella colonna di partenza, solo valori unici. Una volta effettuate tali operazioni, nella colonna "Sepal-length", restano soltanto 3 valori unici ordinati: 4.8, 5.6 e 6.4; i valori duplicati vengono scartati.

Fatto ciò, per ogni valore unico rimasto nella colonna, si va a creare una colonna aggiuntiva con il nome: *"nome-attributo-originale <= valore-unico-continuo"*, andando a sostituire all'espressione generale il nome dell'esempio, si ottiene: **"Sepal-lenght <= 4.8"** e i valori che conterrà saranno solo 0 per no e 1 per si, in questo caso 1 0 0; la prossima colonna avrà nome "Sepal-length <= 5.6" e avrà valori 1 1 0 , l'ultima colonna ovviamente avrà tutti 1, infatti "Sepal-lenght <= 6.4" avrà tutti si come risposta, 1 1 1.

Se una colonna ha 40 valori continui e, dopo averli ordinati in modo ascendente, di unici ne rimangono solo 30, si andranno a creare 30 colonne aggiuntive seguendo la regola che, a questo punto, dovrebbe essere stata ampiamente compresa.

Una volta create le 30 colonne, *si cancella quella originale da cui si sono ricavati i 30 valori unici* e si mantengono solo le 30 create (che avranno tutti 0 e 1 come valori, tranne l'ultima che può essere tranquillamente cancellata contenendo solo 1) insieme alle eventuali colonne di attributi categorici che però non vanno toccate e lasciate così come sono. Esatto, gli *attributi categorici non vengono modificati*, poiché il MDT li processa direttamente e senza alcuna modifica esterna al dataframe.

Le parti di codice che consentono di manipolare il dataframe per rendere il MDT compatibile con gli attributi continui sono disponibili nel Listato seguente:

```
1  # inizio creazione colonne
2  int_features = dataframe.select_dtypes(include=int)
3  float_features = dataframe.select_dtypes(include=float)
4  continuous_features = int_features.columns.tolist() + float_features.columns.tolist()
5  for continuous_feature in continuous_features:
6     X_train[continuous_feature] = X_train[continuous_feature].map(float)
7  cond_var_list = []
8  for feature in continuous_features:
9     training_classes = np.unique(X_train[feature])
10    for i in range(len(training_classes)):
11       attribute_name = feature + " <= " + str(training_classes[i])
12       cond_var_list.clear()
13       for value in feature_values_list:
14          if value <= training_classes[i]:
15             cond_var_list.append(1.0)
16          else:
17             cond_var_list.append(0.0)
18       to_be_added_df = pd.DataFrame({attribute_name: cond_var_list})
19       X_train = X_train.reset_index(drop=True).join(to_be_added_df.reset_index(drop=
             True), how='left')
20    del X_train[feature]
21 # useremo il training set come copia non framentata del training_data
22 training_temp = X_train.copy(deep=True)
23 del X_train
24 X_train = training_temp
```

Listing 3.7: Codice per interpetare gli attributi continui

Nella riga 4 del listato 3.7 viene creata una lista contenente i nomi, quindi stringhe, di tutte le colonne aventi valori continui, nelle righe 2 e 3 riesco a selezionarle tra tutte quante. Tra tutte quante significa che le colonne categoriche non verranno selezionate e quindi non faranno parte della lista.

Il for nella riga 5 serve per convertire tutti i valori a float delle colonne interessate, i cui nomi sono presenti nella lista. Questa conversione serve per avere *solo due tipi di valori nel dataframe*: **numerici**, float di base e interi convertiti a float, e **non numerici**, stringhe di valori, che il Python dichiarerà essere di tipo *object*. *Ogni valore float sarà interpretato come continuo dal MDT, ogni valore object sarà interpretato come categorico.* Nella riga 7 viene creata

una lista che andrà a contenere gli zeri e gli uno per formare le nuove colonne a partire dai valori continui unici. Per ogni valore continuo unico avremo una colonna aggiuntiva.

Nelle righe seguenti vi è finalmente il **codice che crea le colonne**: il for di riga 8 scorre tutti gli attributi continui; nella riga 9, dopo aver effettuato l'ordinamento crescente, si acquisiscono tutti i valori unici appartenenti all'attributo su cui si è in quel giro di for. Nella riga 11 viene creato il nome della nuova colonna a partire dal nome originale dell'attributo (della colonna) e del suo valore unico originale, nel for più interno a riga 13 viene riempita la lista creata alla riga 7 con i valori, 0 e 1, corretti. La riga 18 utilizzerà la lista di riga 7 per creare la colonna vera e propria e aggiungerla al dataframe; infine, la join, a riga 19, eviterà il problema della frammentazione di memoria.

3.6 Visualizzazione del MDT

In questa parte si mostra la stampa testuale per il MDT. Una volta effettuato il training, il codice seguente stamperà sia a schermo che su file, l'albero trainato con tutti i suoi livelli di profondità.

```
1  def displayMDT(self, root, file, width):
2      # definisco il numero di spazi:
3      const = int(root.depth * width ** 1.5)
4      spaces = "-" * const
5      # se sono sul nodo root
6      if root.isRoot:
7          if root.isLeaf:
8              # sono sul nodo root di un albero che non si e' nemmeno potuto formare
9              root_string = "sono root e allo stesso tempo foglia, posso solo darti la
                   classificazione " \
10                         f"del most common value che e' {root.classification}"
11             print(root_string)
12             file.write(root_string)
13         else:
14             # sono la radice di un albero che si sta formando, ergo verro' splittato
15             if self.criterion == 'entropy':
16                 root_string = f"Root at depth:{root.depth} with Split Condition: |{root.split
                       _condition}| " \
17                               f"with info_gain = {root.gain}"
18                 print(root_string)
19                 file.write(root_string)
```

```
20          elif self.criterion == 'gini':
21              root_string = f"Root at depth: {root.depth} with Split Condition: |{root.
                    split_condition}| " \
22                            f"with gini_gain = {root.gain}"
23              print(root_string)
24              file.write(root_string)
25          else:
26              raise Exception("Wrong criteria input, possible choices are: 'entropy' or '
                    gini'")
27      else:
28          # se sono una foglia
29          if root.isLeaf:
30              if is_numeric_dtype(root.split_value):
31                  if root.split_value == 1.0:
32                      word: str = "yes"
33                  elif root.split_value == 0.0:
34                      word: str = "no"
35                  else:
36                      raise Exception("valid values at this point can either be 1.0 or 0.0")
37              else:
38                  word: str = str(root.split_value)
39
40              string_leaf = 'depth:', str(root.depth), f"|{spaces} if {root.parent.split_
                    condition} == " \
41                                            f"{word} --> Classification: {root.classification}"
42              print(string_leaf)
43              file.writelines(string_leaf)
44          # se sono un nodo interno
45          else:
46              if self.criterion == 'entropy':
47                  string_not_leaf = 'depth:', str(root.depth), \
48                                    f"|{spaces} Split Condition: |{root.split_condition}| " \
49                                    f"with info_gain = {root.gain}"
50                  print(string_not_leaf)
51                  file.writelines(string_not_leaf)
52              elif self.criterion == 'gini':
53                  string_not_leaf = 'depth:', str(root.depth), \
54                                    f"|{spaces} Split Condition: |{root.split_condition}| " \
55                                    f"with gini_gain = {root.gain}"
56                  print(string_not_leaf)
57                  file.writelines(string_not_leaf)
58              else:
59                  raise Exception("Wrong criteria input, possible choices are: 'entropy' or '
                        gini'")
60      if len(root.children) > 0:
```

```
61      for child in root.children:
62          assert (child.parent == root)
63          self.displayDT(child, file, width)
```

Listing 3.8: Codice per stampare il MDT su stdout e su file

Alle righe 3 e 4 del Listato 3.8 viene creato un offset, dipendente da parametri configurabili, per generare i livelli di profondità dell'albero. La funzione distungue molto attentamente i casi in cui il calcolo di impurità viene effettuato con l'Entropia o con il Gini index, allo stesso modo si è prestata molta attenzione alla distinzione degli attributi categorici da quelli continui. La funzione *"is_numeric_type()"* della libreria pandas aiuta a raggiungere lo scopo.

Quando si ha un attributo continuo (che sarà sicuramente un float, perché anche se fosse stato originariamente di tipo int, tale int sarebbe stato poi convertito in float, al fine di avere, nell'albero, solo due tipi: *numerico*, cioè float e *non numerico*, cioè object) la funzione *is_numeric_type()* restituirà true, altrimenti dinnanzi ad un attributo categorico restituirà false.

Segue una stampa del MDT dopo aver effettuato la "fit" partendo dai dati di training del dataset **Iris:**

```
Root at depth:0 with Split Condition: |PetalLengthCm <= 1.9| with info_gain = 0.9149263727797274
depth:1|-------- if PetalLengthCm <= 1.9 == yes --> Classification: Iris-setosa
depth:1|-------- Split Condition: |PetalWidthCm <= 1.7| with info_gain = 0.6325253077661508
depth:2|---------------- Split Condition: |PetalLengthCm <= 5.1| with info_gain = 0.1952667279783059
depth:3|------------------------ Split Condition: |SepalWidthCm <= 2.5| with info_gain = 0.10973196301060684
depth:4|------------------------------- Split Condition: |PetalLengthCm <= 4.4| with info_gain = 0.4464393446710155
depth:5|---------------------------------------- if PetalLengthCm <= 4.4 == yes --> Classification: Iris-versicolor
depth:5|---------------------------------------- Split Condition: |SepalLengthCm <= 6.0| with info_gain = 0.91829583405448
depth:6|------------------------------------------------- if SepalLengthCm <= 6.0 == yes --> Classification: Iris-virginica
depth:6|------------------------------------------------- if SepalLengthCm <= 6.0 == no --> Classification: Iris-versicolor
depth:4|------------------------------- if SepalWidthCm <= 2.5 == no --> Classification: Iris-versicolor
depth:3|------------------------ if PetalLengthCm <= 5.1 == no --> Classification: Iris-virginica
depth:2|---------------- Split Condition: |PetalLengthCm <= 4.8| with info_gain = 0.11901271691308317
depth:3|------------------------ Split Condition: |SepalLengthCm <= 5.9| with info_gain = 0.9182958340544896
depth:4|------------------------------- if SepalLengthCm <= 5.9 == yes --> Classification: Iris-versicolor
depth:4|------------------------------- if SepalLengthCm <= 5.9 == no --> Classification: Iris-virginica
depth:3|------------------------ if PetalLengthCm <= 4.8 == no --> Classification: Iris-virginica
accuracy_score = 0.98
f1_score = 0.98
```

Figura 3.1: Esempio di stampa testuale del MDT con dataset Iris

3.7 Le Tre Modalità del MixedDecisionTree

Come già anticipato nelle precedenti sezioni, il MDT presenta tre modalità di funzionamento: **classic, median e binned**; l'utente sceglie la modalità che vuole utilizzare assegnando al parametro self.tree_mode la stringa 'classic, 'median' o 'binned' nel momento in cui istanzia

l'albero. In quel momento può anche scegliere il numero di bins da usare esclusivamente per le modalità median e binned.

3.7.1 Modalità Classic

Funzionamento classico dell'albero: offre la massima precisione possibile per gli attributi continui, dal momento che, data una colonna a valori continui, il dataframe verrà modificato per creare una colonna aggiuntiva per ogni valore continuo non duplicato. Ovviamente tutta questa precisione ha un costo in tempo e spazio notevole.

```
1  def _classic_mode(self, dataframe):
2     dataframe.columns = dataframe.columns.str.replace(' ', '_')
3     dataframe[self.class_name] = dataframe[self.class_name].map(str)
4
5     # riduco le righe del dataset, se questi supera la soglia di cut_threshold
6     if len(dataframe) > self.cut_threshold:
7        dataframe = dataframe.drop(labels=range(self.cut_threshold, len(dataframe)), axis
             =0)
8
9     y = dataframe[self.class_name].map(str)
10    columns = [c for c in dataframe.columns if c != self.class_name]
11    X = dataframe[columns]
12    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33, random_state
          =43, stratify=y)
13
14    # ... righe di codice che creano le colonne...
15
16    X_train = X_train.reset_index(drop=True)
17    y_train = y_train.reset_index(drop=True)
18    return X_train, X_test, y_train, y_test
```

Listing 3.9: Codice della modalità classic

Nel Listato di codice 3.9 la riga 2 elimina ogni spazio tra le parole del nome di un attributo, rimpiazzandolo con un trattino basso; la riga 3 forza la conversione a stringa della variabile dipendente Y (la colonna di classificazione, quella che contiene le etichette di classificazione).

Nella riga 6 è evidente il funzionamento del parametro soglia cut_threshold che è utilizzabile solo in modalità classic, essendo la più dispendiosa in termini di spazio e tempo; infatti, tale parametro ha lo scopo di ridurre il totale delle righe del dataset.

Nella riga 10 vengono selezionate tutte le colonne del dataset, tranne quella di classificazione, e assegnate alla variabile X. La colonna di classificazione invece è stata memorizzata nella variabile Y, che, insieme ad X, daremo in pasto alla funzione di sklearn "train_test_split", la quale restituirà tutti i componenti necessari ad effettuare il training dell'albero con il training set, X_train e y_train, e il testing set con X_test, y_test.

3.7.2 Modalità Median

La modalità median è esattamente uguale alla modalità classic in termini di codice tranne per le seguenti righe:

```
1   # ....
2   y = dataframe[self.class_name].map(str)
3   columns = [c for c in dataframe.columns if c != self.class_name]
4   X = dataframe[columns]
5   X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33, random_state
        =43, stratify=y)
6
7   # attenzione: la cut e la mid vanno applicate SOLO al training set
8   for feature in continuous_features:
9      X_train[feature] = pd.cut(X_train[feature], self.number_of_bins)
10     X_train[feature] = X_train[feature].apply(lambda x: x.mid)
11
12  # ... righe di codice che creano le colonne, omesse
13  # ... perche' presenti nel listato 3.7 a pagina 32
14
15  # resetto gli indici del training_set con drop=True
16  X_train = X_train.reset_index(drop=True)
17  y_train = y_train.reset_index(drop=True)
18
19  return X_train, X_test, y_train, y_test
```

Listing 3.10: Codice della modalità median

La differenza tra il Listato 3.10 della modalità median e quello 3.9 della modalità classic è subito evidente: non è presente codice riguardante il parametro soglia cut_threshold, e, alla riga 9, applico la cut con un dato numero di bins (al momento dell'istanziazione dell'albero viene passato anche il numero di bins da utilizzare per le modalità median e binned) solo al training

set, mentre alla riga 10, sempre al training set, viene applicata la mid tramite una lambda expression.

Queste righe di codice vanno ad alleggerire notevolmente la creazione di colonne che avverrà dopo, esattamente con lo stesso codice della modalità classic, l'unica differenza è che il numero di bins pone un limite superiore a quanti valori unici si avranno nella colonna.

Se, ad esempio, la colonna di valori continui, "Petal-length", possiede 150 valori in virgola mobile (di tipo float che il Python considera come valori continui) tutti diversi tra loro, nella modalità classic avrei dovuto creare 150 colonne, una per ogni valore unico, mentre qui, in modalità median, vado a creare un numero di colonne pari al numero di bins. Questo significa che avrò anche un numero di valori unici pari al numero di bins e questi valori sono stati calcolati per essere uniformemente distribuiti in "numero-di-bins" midpoints.

Questa operazione la applico solo al training set, i valori del test set rimangono immutati. Capiamo quindi quanto questa modalità ci faccia risparmiare in spazio e tempo senza nemmeno perdere troppi punti percentuali in Accuracy, o F1-score, rispetto alla precisissima ma inefficientissima modalità classic, che non pone un limite superiore ai valori unici in ogni colonna.

3.7.3 Modalità Binned

La modalità binned usa anch'essa la pandas.cut, ma senza applicare la mid tramite l'espressione lambda vista alla riga 10 del listato 3.10. Lo scopo di questa modalità è di trasformare una colonna a valori continui, in una colonna di stringhe, dove le stringhe sono intervalli di valori. La precisione degli intervalli è proporzionale al numero di bins. Tale modalità deve essere applicata *anche* al testing set, altrimenti in fase di predizione non avremo alcun match.

Come è possibile notare alla riga 10 dello snippet 3.11 qui in basso, viene applicata la cut a tutto il dataframe, da cui, solo successivamente, si ricaveranno il training set e il testing set tramite la funzione di libreria "train_split_test()".

```
1 def _binned_mode(self, dataframe):
2     dataframe.columns = dataframe.columns.str.replace(' ', '_')
3     dataframe[self.class_name] = dataframe[self.class_name].map(str)
4     int_features = dataframe.select_dtypes(include=int)
5     float_features = dataframe.select_dtypes(include=float)
```

```
6   continuous_features = int_features.columns.tolist() + float_features.columns.tolist()
7
8   # il nucleo della modalita' binned e' tutto qui:
9   for feature in continuous_features:
10     dataframe[feature] = pd.cut(dataframe[feature], self.number_of_bins)
11
12  y = dataframe[self.class_name].map(str)
13  columns = [c for c in dataframe.columns if c != self.class_name]
14  X = dataframe[columns]
15  X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33, random_state
        =43, stratify=y)
16
17  return X_train, X_test, y_train, y_test
```

Listing 3.11: Codice della modalità binned

Nel listato 3.11, le prime righe operano allo stesso identico modo delle altre modalità, vengono selezionate tutte le colonne a valori continui lasciando invariate le colonne a valori categorici, e *si applica la cut sia al training set che al test set*, come appare evidente alla riga 10.

Il risultato di questa applicazione è quello di creare intervalli di valori che il Python vedrà come intervalli di stringhe (di tipo object) e la precisione di tali intervalli dipende dal numero di bins con cui abbiamo istanziato l'albero.

Si è quindi, in modalità binned, trasformato direttamente il continuo in categorico senza la necessità di creare colonne aggiuntive come avviene in modalità classic o median.

Inutile dire che questa è la modalità più economica a livello di risorse come tempo e spazio, però, a differenza di median, che rappresenta il compromesso perfetto tra prestazioni e costi computazionali (sarà evidente nel prossimo capitolo), la modalità binned soffre un po' troppo di perdite in accuracy e f1-score.

Se si dovesse effettuare il training del MDT e si dispone di tempo e spazio illimitati, allora utilizzerei senza dubbi la modalità classic, ma se il tempo e lo spazio sono limitati, userei senza alcuna esitazione la modalità median.

Capitolo 4

Esperimenti

You can have data without information, but you cannot have information without data. (Daniel Keys Moran)

4.1 Informazioni preliminari

Oltre alla completa scrittura del MDT con tutte le sue modalità, è stato necessario scrivere altri programmi al fine di effettuare i test presenti nel seguito di questo capitolo.

Il primo programma denominato **SKLEARN_verify.py** testa l'albero di Sklearn con tutti i dataset contemplati in questa tesi al fine di ricavarne tabelle salvate nella cartella *Results*, contenenti metriche importanti come accuracy, f1-score e tempo di esecuzione al variare di parametri come: *criterio di impurità, modalità dell'albero, cut_threshold, numero di bins, max_depth, min_sample_split e min_sample_leaf.* Tale programma, inoltre, **adatterà le tre modalità** classic, median e binned all'albero di Sklearn, manipolando opportunamente il dataframe *dall'esterno*.

Per quanto concerne la parte prettamente "visuale" dell'albero, il primo programma, oltre a stampare l'albero di Sklearn in versione testuale, lo stamperà anche a livello grafico. Ciò significa che si potranno visualizzare i nodi interni, gli archi, le foglie di classificazione più altre informazioni utili; tali stampe verranno salvate nella cartella apposita denominata *Visual_sklearn*.

Il secondo programma è denominato **MDT_verify.py** ed eseguirà lo stesso tipo di test del primo programma generando le stesse tabelle che verranno salvate nella cartella *Results*. L'unica differenza è che il secondo programma effettuerà stampe solo testuali come nella Figura 3.1 a pagina 42. Tali stampe verranno salvate nella cartella *Visual_mixedDT*

Altri programmi acquisiranno in input le tabelle in formato *.csv* in uscita dal primo e dal secondo programma di test al fine di generare in output tutte le tabelle, gli istogrammi e i grafici sovrapposti che analizzeremo nel corso di questo capitolo.

4.2 Presentazione dei Dataset

I dati sono i veri protagonisti di tutto ciò che riguarda il Machine Learning, ergo è arrivato il momento di conoscere i dataset utilizzati in questa tesi:

DATASET	RIGHE	COLONNE	CLASSI	TIPO
Auto	392	7	3	MISTO
Breast-Cancer	271	8	2	MISTO
Carseats	279	9	2	MISTO
Churn	500	13	2	MISTO
Diabetes	393	8	2	CONTINUI
HouseVotes	434	16	2	CATEGORICI
Iris	150	4	3	CONTINUI
Parkinsons	100	22	2	CONTINUI
Prostate	97	7	2	CONTINUI
Sacramento	932	6	3	MISTO

Figura 4.1: Tutti i datasets contemplati in questa tesi

Il primo dataset considerato è denominato **Auto** e, come la parola stessa suggerisce, serve a classificare le automobili per peso, anno, tipo di cilindrata, tempo di accelerazione da 0 a 100Km/h, potenza del motore in termini di cavalli ecc. È interessante soprattutto perché presenta 3 classi di classificazione e non sempre le solite due.

Il secondo dataset, **Breast-Cancer**, predice, utilizzando numerosi dati clinici, se la persona in questione è portatrice o meno della malattia suggerita dal nome del dataset. Sia quest'ultimo che quello precedente sono datasets misti, cioè compare almeno una colonna categorica e almeno una colonna contenente dati continui.

Carseats è il nome del terzo dataset, quindi si andrà a predire, grazie al nostro albero decisionale, le vendite dei sedili di automobili in base a numerose informazioni, come la pubblicità che è stata effettuata, le vendite precedenti suddivise per zone ecc...

Churn è un altro dataset interessante, consta di 500 righe, come è possibile vedere dalla tabella in Figura 4.1, e si andrà a predire se l'ex cliente, che ha abbandonato un certo servizio fornito da una determinata compagnia, è disposto a rientrarvi ritornando nuovamente cliente.

Tramite il dataset **Diabetes**, come succedeva per "Breast-Cancer", andremo a predire se la persona, con tutti i suoi dati clinici rilevanti al caso, potrebbe incorrere o meno nella malattia descritta dal dataset.

Iris, è praticamente il dataset per antonomasia, citato da molti libri di testo che trattano il nostro argomento, è interessante perché conta 3 classi e non le solite due, è un dataset completamente continuo; quindi, ogni colonna è costituita da valori continui e si andrà a predire, in base a dati come lunghezza del sepalo o lunghezza del petalo, in quale delle tre categorie si colloca l'iris.

Altri due dataset che utilizzano stime mediche sono **Parkinsons** e **Prostate**. Nel primo si andrà a predire se la persona soffre o meno del morbo indicato dal nome del dataset, nel secondo se la persona può avere o meno il cancro alla prostata.

Per concludere si considera un dataset in ambito real estate (immobiliare): **Sacramento**, i dati in esso contenuti sono stati presi da un file di transazioni in un periodo di soli 5 giorni. Le transazioni coinvolgono 932 case (e quindi conta 932 righe) e i loro rispettivi prezzi di vendita nella città di Sacramento.

4.3 Test

In questa sezione verranno effettuati confronti tra il MDT e il DT di sklearn, si avranno a supporto anche tabelle, istogrammi e grafici sovrapposti per una più facile lettura e comprensione.

Tutti i confronti effettuati sono basati sulla variazione di alcuni parametri tenendone costanti altri. I parametri in gioco sono quelli di pre-pruning che già si conoscono, uniti ai criteri di impurità scelti (Entropia o Gini Index) e al numero di bins (20 o 11).

Per prima cosa si è voluta testare la ACCURACY raggiunta dai due alberi decisionali nelle tre modalità *classic, median e binned*, già presentate nel terzo capitolo, in particolari **configurazioni di interesse**. Tali configurazioni si trovano nella tabella di Figura 4.4 a pagina 51.

Ogni configurazione è determinata da una scelta particolare degli *iper-parametri*, che sono *max-depth*, i cui valori appartengono all'insieme $\{2, 3, 4, 5, 6, 7, 8, 25, 50\}$, *min-sample-split*, i cui valori appartengono all'insieme $\{2, 3, 5, 10, 25, 50\}$, e *min-sample-leaf*, i cui valori possibili appartengono all'insieme $\{1, 3, 5, 10, 25, 50\}$, da una scelta circa il *criterio di impurità* utilizzato (che può essere l'Entropia o il Gini index) e da una scelta circa il *numero di bins* utilizzati.

Si vedranno più dettagliatamente anche grafici sovrapposti per ognuna delle configurazioni considerate, tali grafici sono disponibili a pagina 52 e 53

Nella prima tabella di Figura 4.2 si ha, per ogni dataset, una configurazione privilegiata in cui è possibile studiare l'andamento della accuracy al variare dei parametri menzionati sopra. Seguiranno istogrammi che visivamente agevoleranno la lettura delle tabelle stesse.

La seconda tabella in Figura 4.6 a pagina 56, è sprovvista di configurazioni di particolare interesse, infatti essa mostra come varia la *massima* ACCURACY per ogni dataset a prescindere da particolari configurazioni di parametri. Quindi, per ogni dataset, registriamo, in corrispondenza di ogni modalità, il valore di picco in ACCURACY per il MDT e per quello di sklearn.

Seguiranno poi, mantenendo lo stesso ordine, le tabelle, gli istogrammi e i grafici sovrapposti per la seconda metrica presa in considerazione in questa tesi: l'F1_SCORE.

4.3.1 Metrica considerata: ACCURACY

	ACCURACY					
	Classic		Median		Binned	
DATASET	MDT	SKL	MDT	SKL	MDT	SKL
Auto	0,7385	0,7538	0,6923	0,7154	0,6231	0,7154
Breast-Cancer	0,6556	0,6778	0,6556	0,7000	0,6556	0,7222
Carseats	0,9355	0,9355	0,9032	0,8817	0,8710	0,8495
Churn	0,8424	0,8242	0,7939	0,7818	0,6061	0,7697
Diabetes	0,8077	0,8000	0,7000	0,7000	0,6615	0,7077
HouseVotes	0,9583	0,9583	0,9583	0,9583	0,9583	0,9583
Iris	0,9800	0,9800	0,9800	0,9800	0,9200	0,9200
Parkinsons	0,7576	0,7273	0,8182	0,6700	0,7576	0,6700
Prostate	0,8182	0,7879	0,7879	0,7879	0,7879	0,6970
Sacramento	0,9318	0,9351	0,9351	0,9351	0,9253	0,9318
MEDIA	***0,8426***	***0,8380***	***0,8224***	***0,8110***	***0,7766***	***0,7942***
STD	***0,1074***	***0,1068***	***0,1175***	***0,1182***	***0,1357***	***0,1101***
WINS	**7**	**6**	**8**	**7**	**5**	**7**

Figura 4.2: Confronti su metrica Accuracy in tutte le modalità

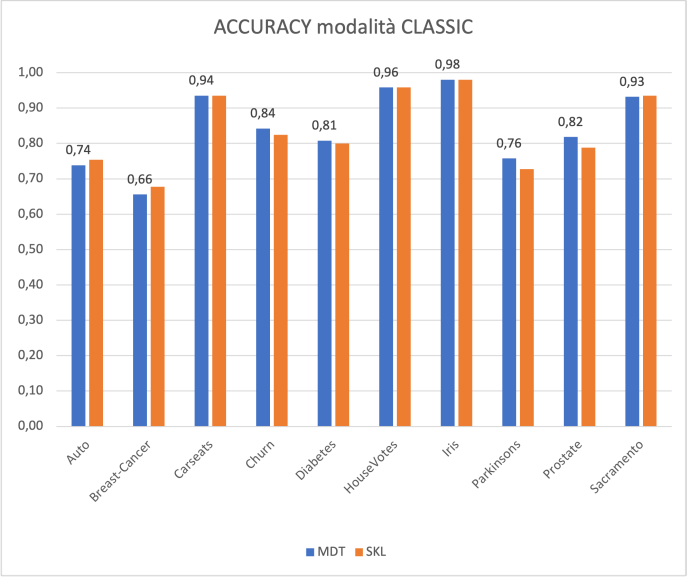
La tabella in Figura 4.2 fornisce una panoramica abbastanza esaustiva di come il MDT e l'albero di Sklearn performano in ACCURACY a parità di configurazione. Tali configurazioni sono consultabili nella tabella a pagina 51. Ad esempio, per il dataset "Auto", si è scelta come configurazione di particolare interesse la seguente: Impurità = Gini Index, Numero di Bins = 20, max_depth = 4, min_sample_leaf = 5 e l'iper-parametro di pre-pruning min_sample_split che varia. Il fatto che min-sample-split vari si comprenderà meglio osservando i grafici sovrapposti a pagina 52 e 53.

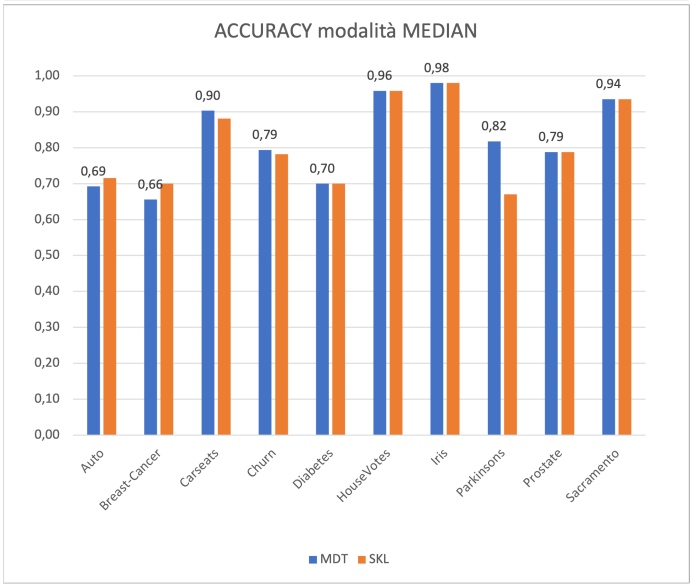
In verde si sono sottolineate le celle dove si verifica una vittoria di un albero rispetto all'altro o un pareggio (un pareggio assegna una vittoria a ciascun albero). Un albero "vince" rispetto all'altro quando, a parità di modalità, performa meglio rispetto all'altro nella metrica che si sta considerando.

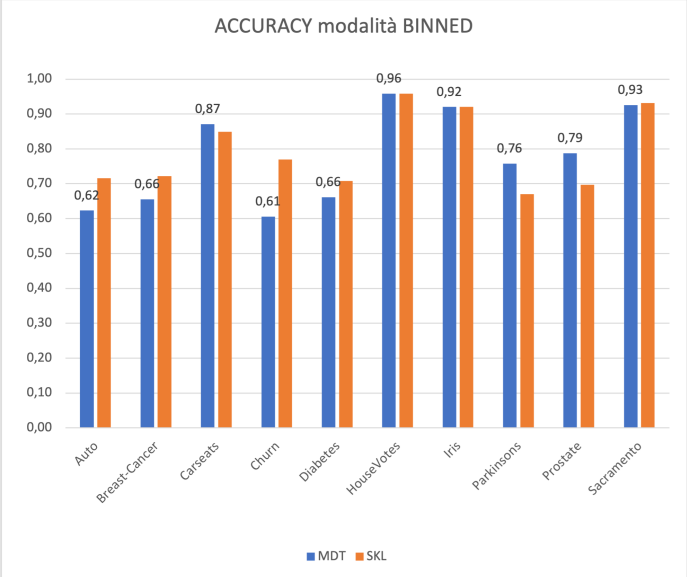
Le ultime righe della tabella contengono la media, la deviazione standard e le vittorie del MDT contro quello di sklearn.

La modalità classic, come si vedrà in tutti i vari confronti, è quella implementata per fornire la massima ACCURACY possibile pagata però a caro prezzo in termini di spazio e tempo, invece, la modalità median è capace in certi casi di eguagliare la precisione della modalità classic ma ad un costo computazionale molto più contenuto. La modalità binned invece è quella più grossolana, è ancora più leggera computazionalmente della modalità median, ma produce performance di gran lunga inferiori ad essa.

Per sottolineare i confronti e vederli in modo più chiaro e comprensibile, si allegano i seguenti grafici che mostrano il confronto diretto tra i due alberi, a parità di configurazione e modalità:


ACCURACY modalità CLASSIC
1,00
0,90
0,80
0,70
0,60
0,50
0,40
0,30
0,20
0,10
0,00
0,74
0,66
0,94
0,84
0,81
0,96
0,98
0,76
0,82
0,93
Auto
Breast-Cancer
Carseats
Churn
Diabetes
HouseVotes
Iris
Parkinsons
Prostate
Sacramento
MDT
SKL


ACCURACY modalità MEDIAN
1,00
0,90
0,80
0,70
0,60
0,50
0,40
0,30
0,20
0,10
0,00
0,69
0,66
0,90
0,79
0,70
0,96
0,98
0,82
0,79
0,94
Auto
Breast-Cancer
Carseats
Churn
Diabetes
HouseVotes
Iris
Parkinsons
Prostate
Sacramento
MDT
SKL


ACCURACY modalità BINNED
1,00
0,90
0,80
0,70
0,60
0,50
0,40
0,30
0,20
0,10
0,00
0,62
0,66
0,87
0,61
0,66
0,96
0,92
0,76
0,79
0,93
Auto
Breast-Cancer
Carseats
Churn
Diabetes
HouseVotes
Iris
Parkinsons
Prostate
Sacramento
MDT
SKL

Volendo tirare le somme sulle vittorie totali, sulla media aritmetica complessiva e sulla deviazione standard totale, si allega la seguente piccola tabella:

	MDT	SKLEARN
MEDIA	0,8139	0,8144
STD	0,0143	0,0058
WINS	20	20

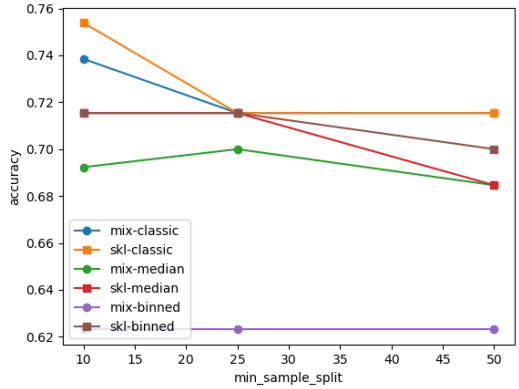
Figura 4.3: Confronti finali tra il MDT e l'albero di Sklearn

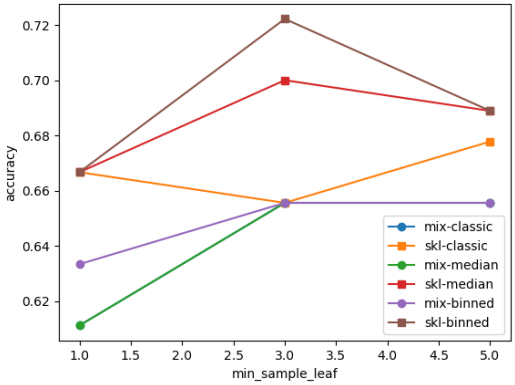
Dalla tabella 4.3 si nota come la deviazione standard totale sia di gran lunga maggiore sul MDT a differenza del DT di sklearn, ciò è dovuto agli scarsi risultati di ACCURACY riportati dal MDT in corrispondenza della modalità binned. Infatti dalla tabella 4.2 a pagina 48 si nota, per ogni dataset, un valore piuttosto deludente di ACCURACY per il MDT nella modalità binned. Tali valori tendono a discostarsi molto dalla media aritmetica facendo aumentare la deviazione standard.

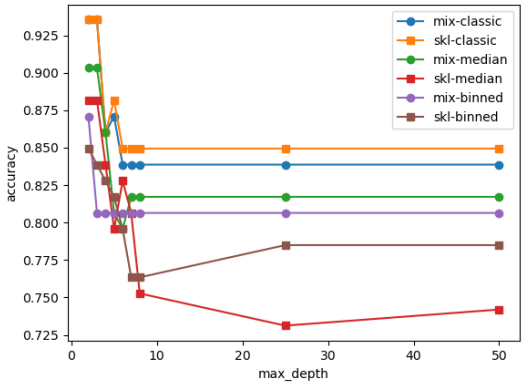
Si è detto che ogni dataset aveva una sua configurazione "privilegiata" di iper-parametri, quindi fissandone due, il terzo poteva variare, cioè assumere più valori in un determinato range. È possibile osservare qui sotto la tabella che descrive le varie configurazioni:

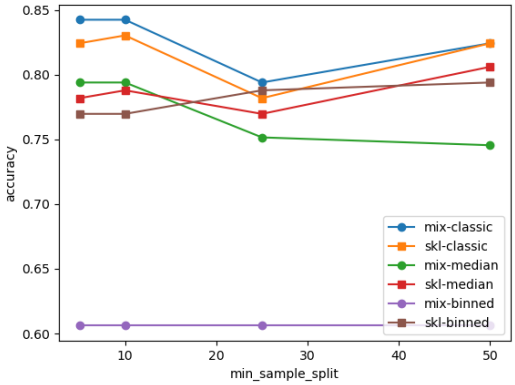
	ACCURACY				
DATASET	IMPURITÀ	BINS	DEPTH	MSS	MSL
Auto	Gini	20	4	varia	5
Breast-Cancer	Entropia	11	25	10	varia
Carseats	Entropia	11	varia	2	1
Churn	Entropia	11	6	varia	3
Diabetes	Gini	20	varia	2	1
HouseVotes	Gini	20	varia	5	3
Iris	Entropia	20	varia	2	1
Parkinsons	Gini	11	varia	3	1
Prostate	Gini	11	7	varia	3
Sacramento	Entropia	20	varia	3	1

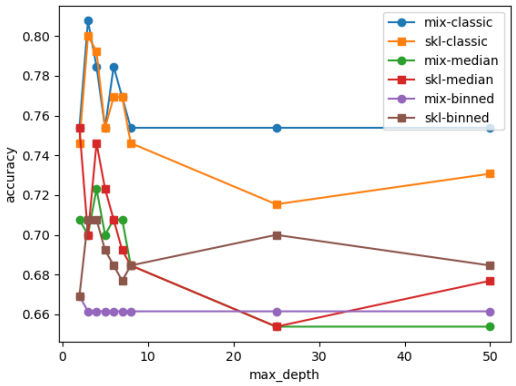
Figura 4.4: Le configurazioni adottate per testare alcuni scenari di ACCURACY e F1_SCORE

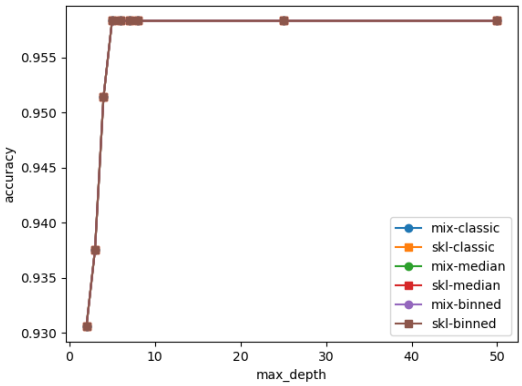

accuracy
min_sample_split
mix-classic
skl-classic
mix-median
skl-median
mix-binned
skl-binned


accuracy
min_sample_leaf
mix-classic
skl-classic
mix-median
skl-median
mix-binned
skl-binned


accuracy
max_depth
mix-classic
skl-classic
mix-median
skl-median
mix-binned
skl-binned


accuracy
min_sample_split
mix-classic
skl-classic
mix-median
skl-median
mix-binned
skl-binned


accuracy
max_depth
mix-classic
skl-classic
mix-median
skl-median
mix-binned
skl-binned


accuracy
max_depth
mix-classic
skl-classic
mix-median
skl-median
mix-binned
skl-binned

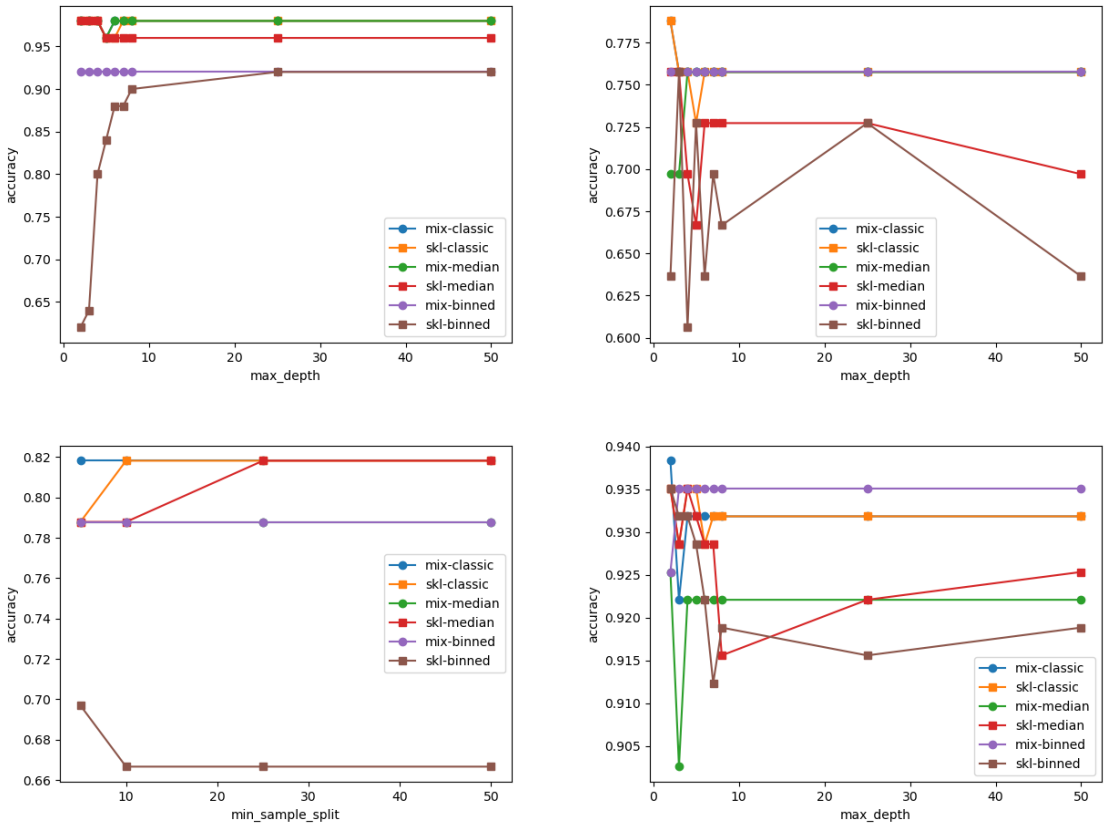


Figura 4.5: Ad ogni dataset, segue lo stesso ordine della tabella in Figura 4.2 a pagina 48, è associata una determinata configurazione di iper-parametri. In ascissa vi è il parametro che varia, in ordinata la metrica scelta per i confronti

Si considerino i grafici sovrapposti in Figura 4.5 in cui ogni grafico rappresenta un dataset seguendo l'ordine della tabella di Figura 4.1.

In ognuno di essi viene mostrato l'andamento della ACCURACY al variare di un qualche parametro di pre-pruning, ricordo che essi sono i seguenti: *max-depth*, *min-sample-split* e *min-sample-leaf* con la particolarità che ogni valore di min-sample-split, affinché il modello abbia senso, deve essere strettamente maggiore del corrispondente valore di min-sample-leaf, cioè non si può avere mss = 3 e msl = 5.

Partendo dal primo grafico sovrapposto relativo al dataset "Auto", esso mostra i valori dell'ACCURACY al variare del parametro min-sample-split mantenendo costanti tutti gli altri parametri su cui basiamo i nostri confronti (parametri di pre-pruning, criterio di impurità e

numero di bins), in questo caso sono fissati i valori di max-depth a 4, min-sample-leaf a 5 e il numero di Bins a 20. Il criterio di impurità scelto è il Gini Index.

Per questo grafico hanno senso, come già detto sopra, solo i valori di min-sample-split strettamente maggiori di quelli di min-sample-leaf, quindi se msl è fissato a 5, i valori su cui potrà variare min-sample-split sono 10, 25 e 50, e infatti, osservando il primo grafico sovrapposto, abbiamo solo tre punti per ogni curva rappresentante le tre modalità per i due alberi, per un totale di sei curve alle quali è associato un singolo colore.

Ad ognuno dei sei colori è associata una modalità di uno dei due alberi, ad es. per il MDT abbiamo le colorazioni azzurro, verde e viola che simboleggiano rispettivamente la modalità classic, median e binned; per quanto invece riguarda il DT di sklearn, ad esso sono associati i colori arancione, rosso e marrone per le stesse modalità.

Sempre osservando il primo grafico risulta evidente come la modalità binned sia gestita male dal MDT (la curva viola), infatti essa è portatrice del valore più basso di accuracy in assoluto, mentre la stessa modalità applicata al DT di sklearn performa abbastanza bene addirittura superando la modalità median del MDT. Per quanto riguarda invece la modalità classic, possiamo notare un andamento simile da parte di entrambi gli alberi che raggiungono pressoché lo stesso punteggio.

Anche la configurazione scelta per il secondo grafico, che rappresenta il dataset Breast-Cancer, in cui a variare è min-sample-leaf, è molto particolare: infatti viene mostrata una vittoria schiacciante da parte della modalità binned applicata però al DT di sklearn (la curva di color marrone), che addirittura supera persino la propria modalità median e classic. Notiamo anche in questo caso la presenza di soli tre punti di variazione.

Il terzo grafico relativo al dataset Carseats, ci illustra la situazione quando a variare è max-depth tenendo fissi i valori di mss e msl. In questo caso variando max-depth e tenendo mss a 2 a msl a 1 otteniamo nove punti di variazione per le curve. La particolarità del terzo grafico è che la modalità median del DT di sklearn performa peggio della modalità binned sul MDT, che si è ormai capito essere il suo punto debole.

Abbastanza curiosa è la situazione nel sesto grafico sovrapposto rappresentante il dataset HouseVotes, in cui variando max-depth, si ottengono tutte e sei le curve con lo stesso andamento. Si nota immediatamente come esse siano totalmente sovrapposte l'una sull'altra.

Il settimo grafico relativo al famosissimo dataset Iris raffigurato a pagina 53 mostra la tipica situazione prevedibile in cui la modalità median e classic coincidono, risultando in alti valori di ACCURACY mentre la modalità binned è la solita a performare peggio sia per il MDT che per il DT di sklearn.

Il nono grafico relativo al dataset Prostate segue la falsariga del settimo, in cui le modalità binned sono i fanalini di coda, in particolare questa volta però quella del DT di sklearn che raggiunge valori davvero bassi, mentre le modalità classic e median performano ottimamente per entrambi gli alberi decisionali.

Il decimo grafico riguardante il dataset Sacramento mostra una rivincita da parte del MDT, infatti la sua modalità binned raggiunge i valori più alti di ACCURACY, se si tralascia il picco più alto raggiunto in un solo punto dalla modalità classic sempre del MDT. La modalità median e binned del DT di sklearn sono quelle che performano peggio raggiungendo i valori più bassi della metrica ACCURACY.

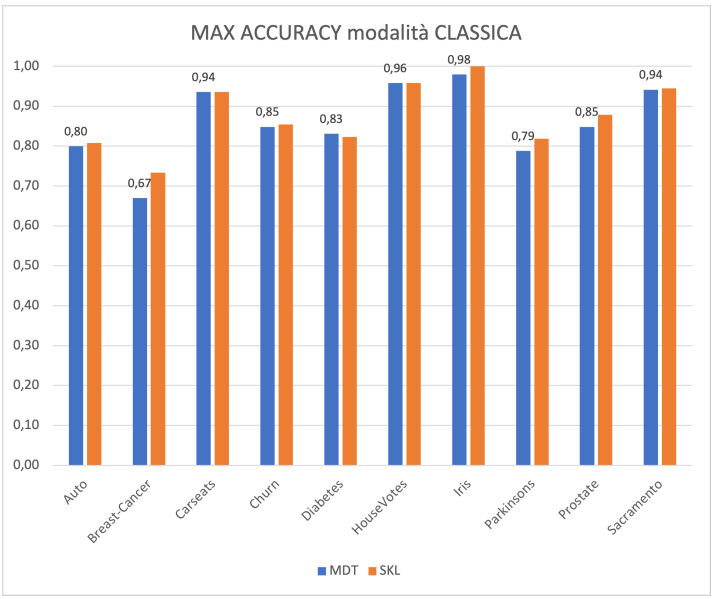
Arriva adesso la parte che potrebbe incuriosire maggiormente il lettore: dati gli stessi datasets, anziché scegliere una configurazione specifica per ognuno di loro, si considerano solamente i massimi assoluti per ogni modalità. La tabella in Figura 4.6 a pagina seguente mostra i risultati ottenuti dal confronto:

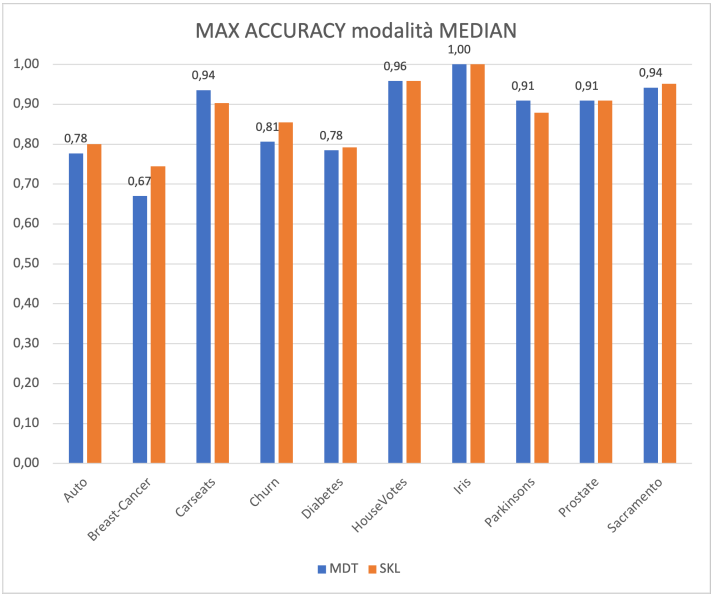
	MAX ACCURACY					
	Classic		Median		Binned	
DATASET	MDT	SKL	MDT	SKL	MDT	SKL
Auto	0,8000	0,8077	0,7769	0,8000	0,7154	0,8231
Breast-Cancer	0,6700	0,7333	0,6700	0,7444	0,6700	0,7444
Carseats	0,9355	0,9355	0,9355	0,9032	0,8817	0,8710
Churn	0,8485	0,8545	0,8061	0,8545	0,7212	0,8485
Diabetes	0,8308	0,8231	0,7846	0,7923	0,7385	0,8231
HouseVotes	0,9583	0,9583	0,9583	0,9583	0,9583	0,9583
Iris	0,9800	1,0000	1,0000	1,0000	0,9600	0,9800
Parkinsons	0,7879	0,8182	0,9091	0,8788	0,7576	0,8182
Prostate	0,8485	0,8788	0,9091	0,9091	0,8182	0,9091
Sacramento	0,9416	0,9448	0,9416	0,9513	0,9351	0,9513
MEDIA	***0,8601***	***0,8754***	***0,8691***	***0,8792***	***0,8156***	***0,8727***
STD	***0,0908***	***0,0788***	***0,0986***	***0,0775***	***0,1046***	***0,0716***
WINS	**3**	**9**	**5**	**8**	**2**	**9**

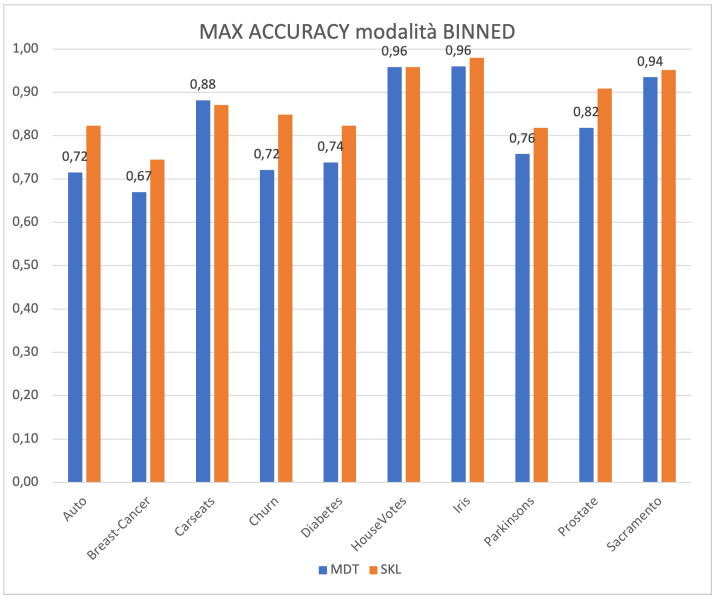
Figura 4.6: Per ogni modalità si considerano solo i massimi per l'accuracy

Da notare, come accadeva per la tabella precedente (a pagina 48) che considerava le configurazioni, le celle in verde significano che l'albero x, a parità di modalità, ha una accuracy più alta rispetto all'albero y. Inutile dire che il MDT in questa tabella, che considera solo i massimi, soffre un po', però, come il lettore potrà subito notare, nella maggior parte dei casi, la differenza è davvero minima.

Seguono gli istogrammi che agevolano maggiormente il confronto diretto tra i due alberi decisionali misurando la massima accuracy nelle varie modalità:


MAX ACCURACY modalità CLASSICA
1,00
0,90
0,80
0,70
0,60
0,50
0,40
0,30
0,20
0,10
0,00
0,80
0,67
0,94
0,85
0,83
0,96
0,98
0,79
0,85
0,94
Auto
Breast-Cancer
Carseats
Churn
Diabetes
HouseVotes
Iris
Parkinsons
Prostate
Sacramento
MDT
SKL


MAX ACCURACY modalità MEDIAN
1,00
0,90
0,80
0,70
0,60
0,50
0,40
0,30
0,20
0,10
0,00
0,78
0,67
0,94
0,81
0,78
0,96
1,00
0,91
0,91
0,94
Auto
Breast-Cancer
Carseats
Churn
Diabetes
HouseVotes
Iris
Parkinsons
Prostate
Sacramento
MDT
SKL


MAX ACCURACY modalità BINNED
1,00
0,90
0,80
0,70
0,60
0,50
0,40
0,30
0,20
0,10
0,00
0,72
0,67
0,88
0,72
0,74
0,96
0,96
0,76
0,82
0,94
Auto
Breast-Cancer
Carseats
Churn
Diabetes
HouseVotes
Iris
Parkinsons
Prostate
Sacramento
MDT
SKL

Le differenze più grandi si hanno in corrispondenza della modalità binned, come è facile notare dalla tabella di Figura 4.6 e dagli istogrammi che la seguono. Nelle altre modalità, invece, le differenze sono minime e, in alcuni casi, il MDT supera anche in ACCURACY l'albero di Sklearn, ad esempio nel Dataset "Carseats" la modalità median del MDT supera di ben 0.323 la accuracy riportata dal DT di sklearn, infatti risulta la prima pari a 0.9355 e la seconda pari a 0.9032

Segue la tabella di Figura 4.7 che riporta il numero di vittorie totali del MDT rispetto al DT di sklearn (e qui si vede come quest'ultimo sia nettamente vincitore conquistando 26 vittorie contro le sole 10 vittorie del MDT), medie complessive e deviazioni standard totali per i due alberi decisionali.

Testando la MAX ACCURACY, si ottiene una deviazione standard totale maggiore per il MDT rispetto ai valori riportati dal DT di sklearn. Anche in questo caso possiamo attribuire la colpa alla modalità binned poco performante sul MDT.

	MDT	SKLEARN
MEDIA	**0,8483**	**0,8758**
STD	**0,0069**	**0,0038**
WINS	**10**	**26**

Figura 4.7: Media delle medie e deviazioni standard totali per i due alberi in max accuracy

Si pongano adesso a diretto confronto gli istogrammi raffigurati a pagina successiva relativi alle due tabelle principali esaminate sino ad ora, la prima in Figura 4.2 a pagina 48, in cui ogni dataset aveva una propria configurazione di parametri, e la seconda in Figura 4.6 a pagina 56, in cui viene mostrata solo la massima accuracy. Per semplicità si sceglie solo la modalità più rappresentativa, quindi quella classic per entrambi.

Si nota subito di quanto le differenze siano minime, ne consegue che non importa quali configurazioni particolari si possano scegliere, i valori di accuracy tenderanno sempre al loro picco, salvo pochi casi dove la differenza non è trascurabile, ad es. nel dataset "Auto" la tabella con le configurazioni raggiunge un risultato più basso di 0.6 punti rispetto a quella dei massimi per entrambi gli alberi.

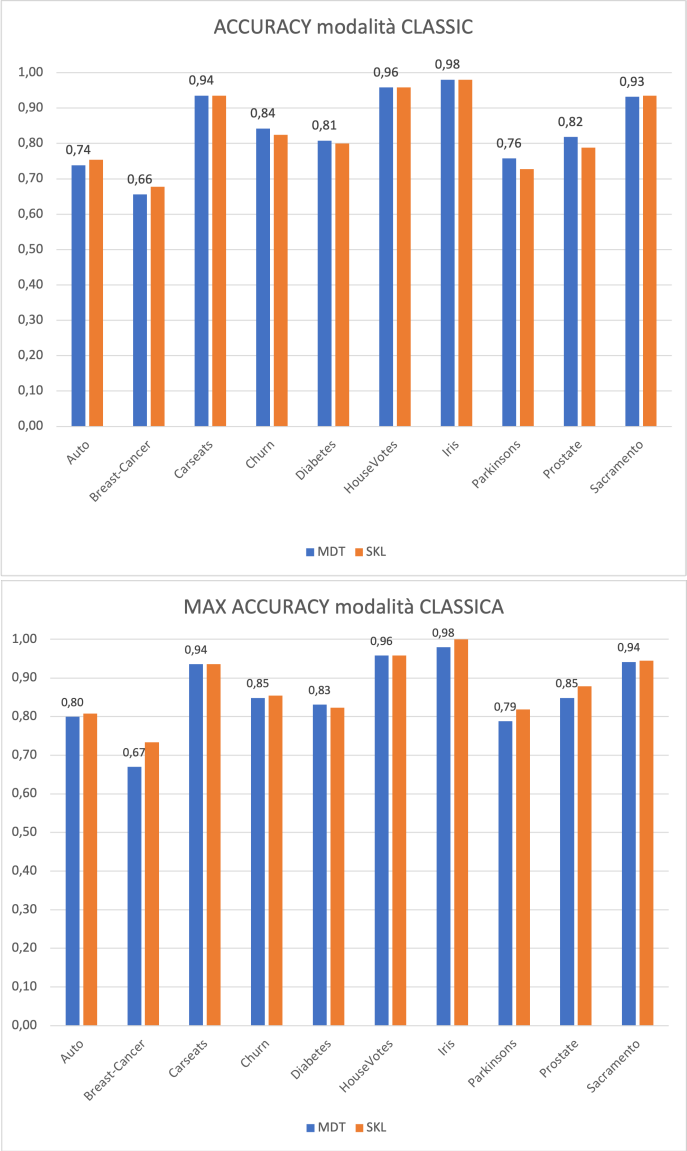


Figura 4.8: Gli istogrammi di cui sopra mostrano il confronto diretto tra la modalità classica dove ogni dataset aveva una configurazione privilegiata e quella in cui contano solo i massimi in ogni modalità

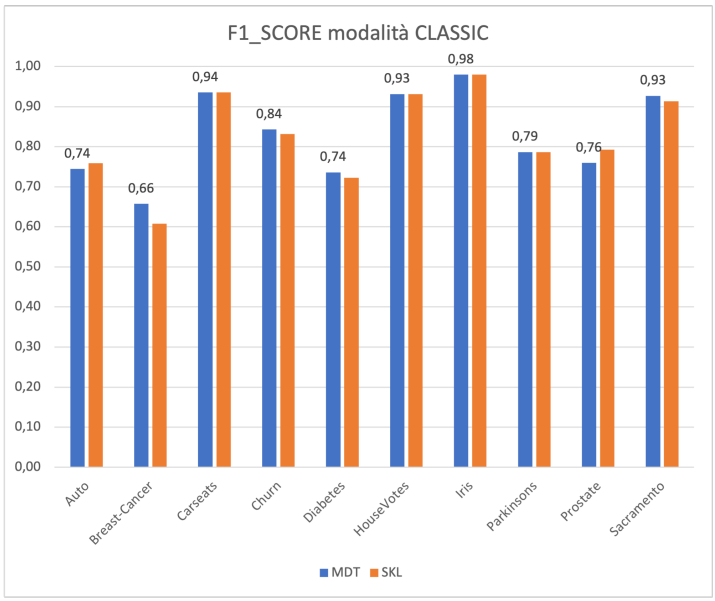
4.3.2 Metrica considerata: F1_SCORE

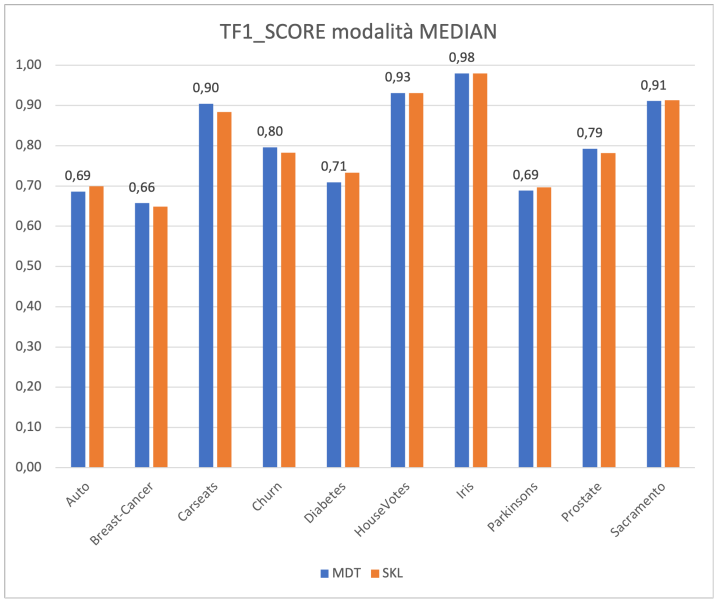
Sarà rifatto tutto quello che è stato fatto fino ad ora, cioè quando la metrica considerata era l'ACCURACY, a tal proposito si utilizzeranno le stesse configurazioni utilizzate precedentemente e si arriverà alla tabella che contempla solo i massimi valori di F1_SCORE.

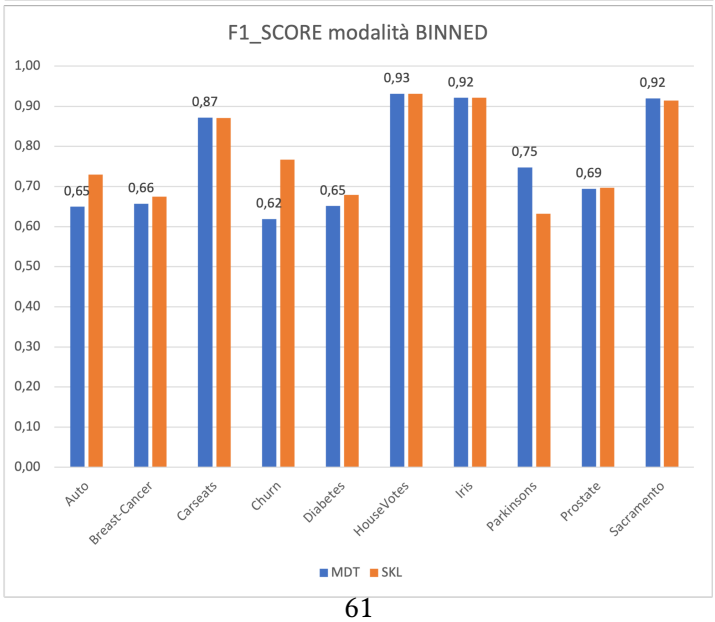
Come si potrà notare osservando tutte le tabelle e i grafici basati sulla metrica F1_SCORE, essi seguono, con piccolissime variazioni, gli stessi andamenti che abbiamo già visto per l'ACCURACY, ergo si è scelto di esonerare il lettore da descrizioni appesantite e ridondanti confidando che tabelle, istogrammi e grafici (che vengono proposti tutti allo stesso modo) siano di immediata comprensione.

	F1_SCORE					
	Classic		Median		Binned	
DATASET	**MDT**	**SKL**	**MDT**	**SKL**	**MDT**	**SKL**
Auto	0,7446	0,7585	0,6861	0,6995	0,6495	0,7296
Breast-Cancer	0,6573	0,6076	0,6573	0,6489	0,6573	0,6749
Carseats	0,9355	0,9355	0,9043	0,8838	0,8720	0,8710
Churn	0,8428	0,8314	0,7962	0,7828	0,6191	0,7666
Diabetes	0,7361	0,7227	0,7093	0,7328	0,6513	0,6789
HouseVotes	0,9312	0,9312	0,9312	0,9312	0,9312	0,9312
Iris	0,9800	0,9800	0,9800	0,9800	0,9211	0,9210
Parkinsons	0,7867	0,7867	0,6884	0,6964	0,7478	0,6323
Prostate	0,7594	0,7929	0,7929	0,7817	0,6944	0,6970
Sacramento	0,9270	0,9130	0,9114	0,9130	0,9196	0,9146
MEDIA	***0,8301***	***0,8260***	***0,8057***	***0,8050***	***0,7663***	***0,7817***
STD	***0,1087***	***0,1154***	***0,1186***	***0,1145***	***0,1299***	***0,1163***
WINS	**8**	**6**	**6**	**6**	**5**	**6**

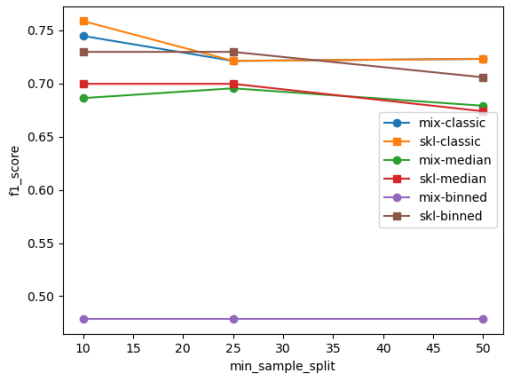
Figura 4.9: Confronti su metrica F1_SCORE sulle stesse configurazioni viste in Figura 4.2

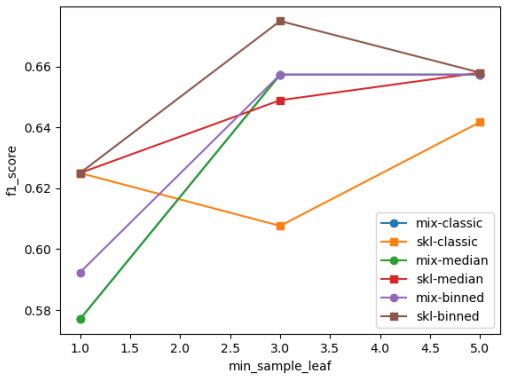

F1_SCORE modalità CLASSIC
1,00
0,90
0,80
0,70
0,60
0,50
0,40
0,30
0,20
0,10
0,00
0,74
0,66
0,94
0,84
0,74
0,93
0,98
0,79
0,76
0,93
Auto
Breast-Cancer
Carseats
Churn
Diabetes
HouseVotes
Iris
Parkinsons
Prostate
Sacramento
MDT
SKL

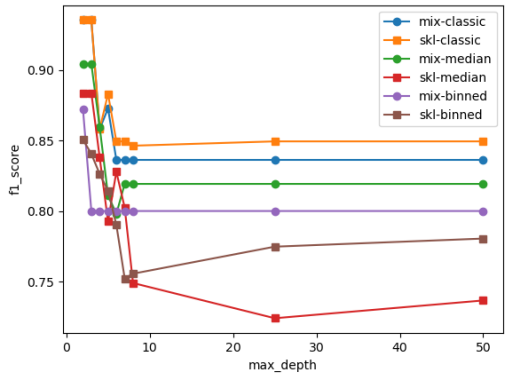

TF1_SCORE modalità MEDIAN
1,00
0,90
0,80
0,70
0,60
0,50
0,40
0,30
0,20
0,10
0,00
0,69
0,66
0,90
0,80
0,71
0,93
0,98
0,69
0,79
0,91
Auto
Breast-Cancer
Carseats
Churn
Diabetes
HouseVotes
Iris
Parkinsons
Prostate
Sacramento
MDT
SKL

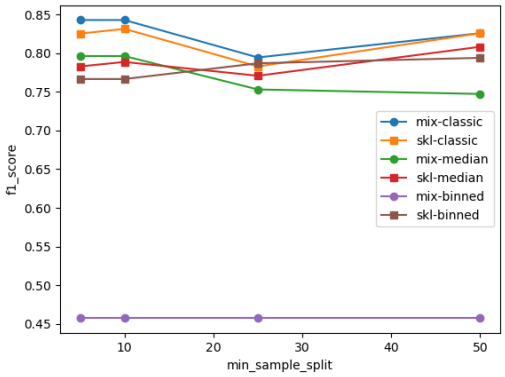

F1_SCORE modalità BINNED
1,00
0,90
0,80
0,70
0,60
0,50
0,40
0,30
0,20
0,10
0,00
0,65
0,66
0,87
0,62
0,65
0,93
0,92
0,75
0,69
0,92
Auto
Breast-Cancer
Carseats
Churn
Diabetes
HouseVotes
Iris
Parkinsons
Prostate
Sacramento
MDT
SKL

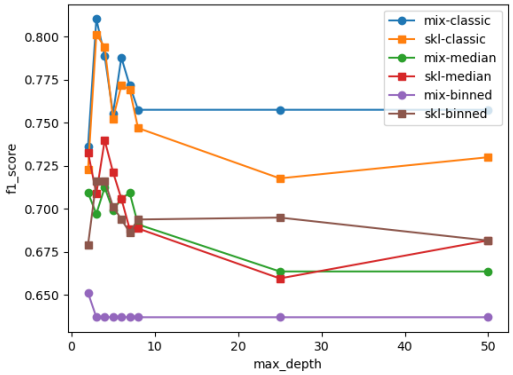
Le configurazioni scelte sono le stesse che si avevano quando la metrica considerata era l'accuracy, tali configurazioni si possono trovare nella tabella di Figura 4.4 a pagina 51 e prevedono un iper-parametro che varia, mentre si mantengono costanti gli altri due. I seguenti grafici mostrano la situazione:

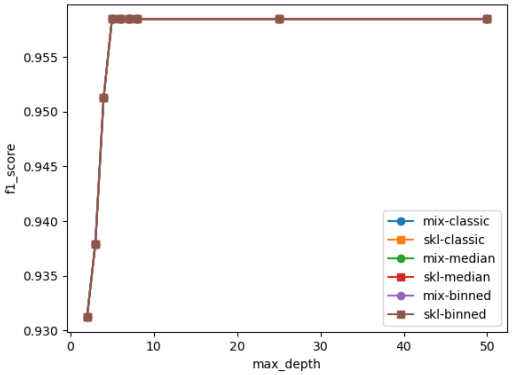












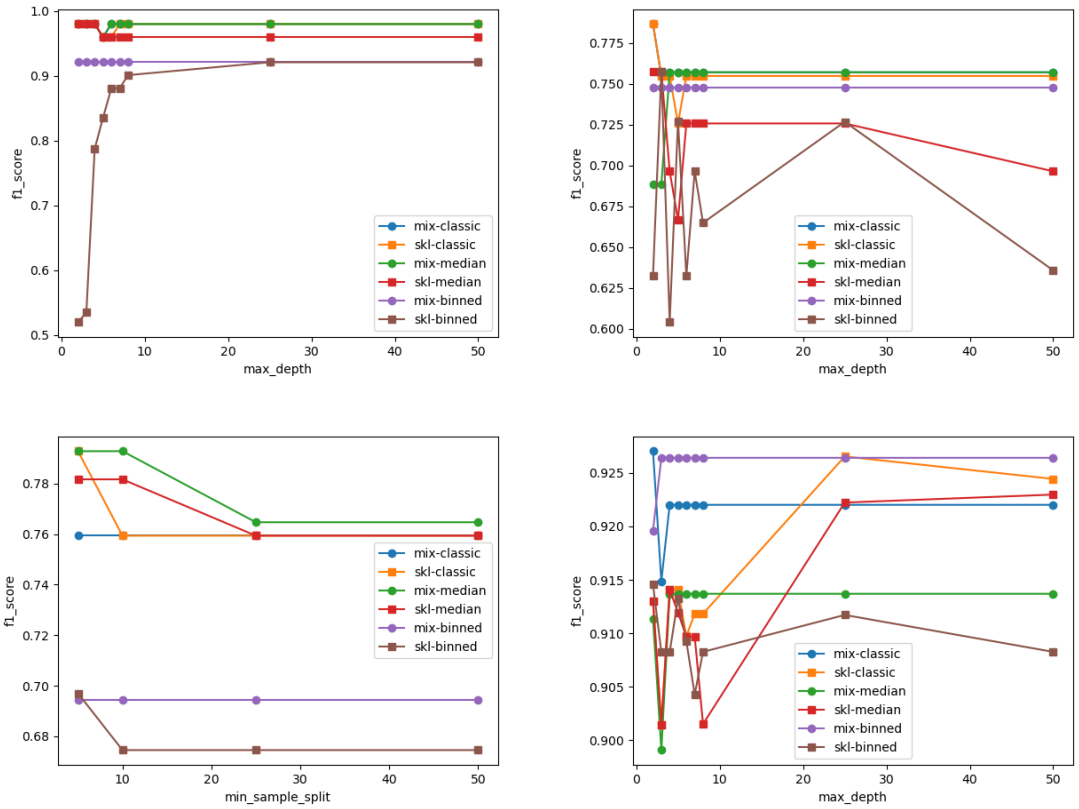


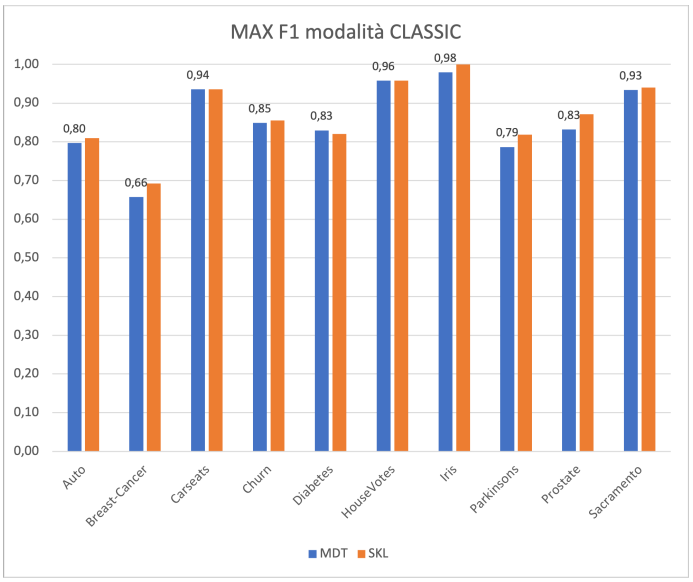
Figura 4.10: Ad ogni dataset, segue lo stesso ordine della tabella in Figura 4.2, è associata una determinata configurazione di iper-parametri. In ascissa vi è il parametro che varia in ordinata la metrica su cui sono basati tutti i confronti

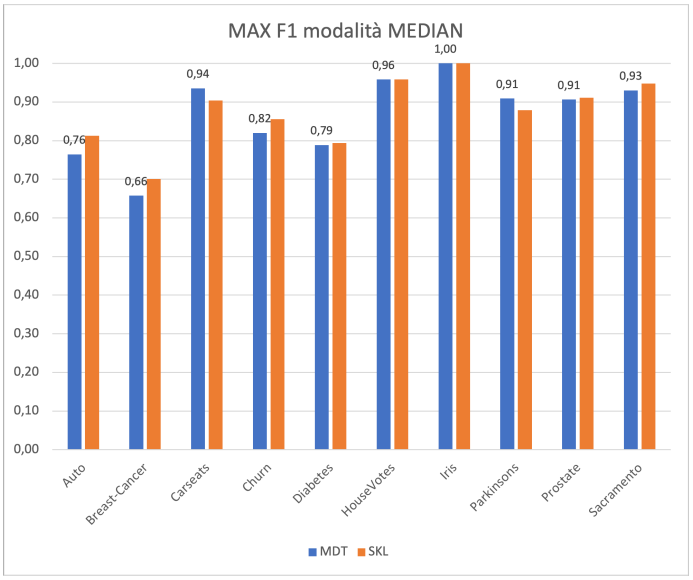
Nella seguente tabella verranno considerati soltanto i picchi, cioè i massimi assoluti di F1_SCORE per ogni modalità.

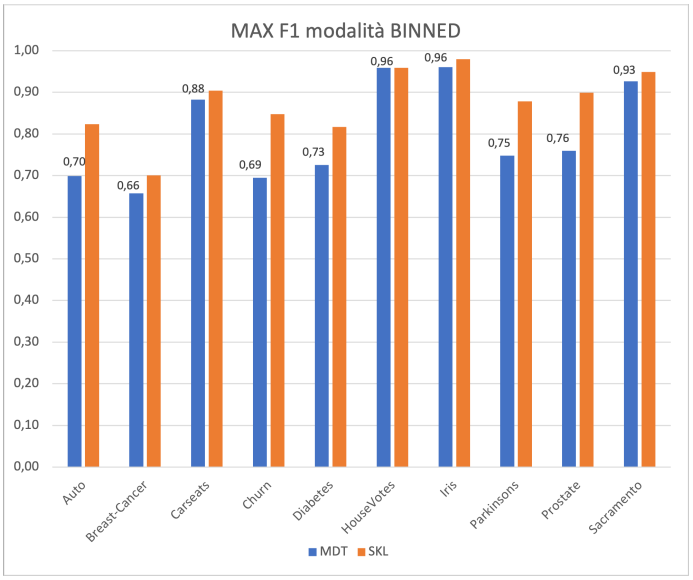
	MAX F1_SCORE					
	Classic		Median		Binned	
DATASET	MDT	SKL	MDT	SKL	MDT	SKL
Auto	0,7969	0,8096	0,7645	0,8128	0,6986	0,8233
Breast-Cancer	0,6573	0,6922	0,6573	0,7009	0,6573	0,7009
Carseats	0,9355	0,9355	0,9355	0,9043	0,8822	0,9043
Churn	0,8490	0,8549	0,8202	0,8552	0,6949	0,8478
Diabetes	0,8297	0,8201	0,7886	0,7940	0,7257	0,8171
HouseVotes	0,9585	0,9585	0,9585	0,9585	0,9585	0,9585
Iris	0,9800	1,0000	1,0000	1,0000	0,9606	0,9800
Parkinsons	0,7867	0,8182	0,9089	0,8786	0,7478	0,8786
Prostate	0,8320	0,8709	0,9065	0,9112	0,7594	0,8992
Sacramento	0,9343	0,9401	0,9301	0,9481	0,9264	0,9486
MEDIA	***0,8560***	***0,8700***	***0,8670***	***0,8764***	***0,8011***	***0,8758***
STD	***0,0986***	***0,0910***	***0,1059***	***0,0890***	***0,1179***	***0,0829***
WINS	**4**	**9**	**4**	**8**	**1**	**10**

Figura 4.11: Confronti su metrica Max F1_SCORE

Per una migliore facilità di lettura, seguono gli istogrammi delle tre modalità per un più facile confronto diretto:


MAX F1 modalità CLASSIC
1,00
0,90
0,80
0,70
0,60
0,50
0,40
0,30
0,20
0,10
0,00
0,80
0,66
0,94
0,85
0,83
0,96
0,98
0,79
0,83
0,93
Auto
Breast-Cancer
Carseats
Churn
Diabetes
HouseVotes
Iris
Parkinsons
Prostate
Sacramento
MDT
SKL


MAX F1 modalità MEDIAN
1,00
0,90
0,80
0,70
0,60
0,50
0,40
0,30
0,20
0,10
0,00
0,76
0,66
0,94
0,82
0,79
0,96
1,00
0,91
0,91
0,93
Auto
Breast-Cancer
Carseats
Churn
Diabetes
HouseVotes
Iris
Parkinsons
Prostate
Sacramento
MDT
SKL


MAX F1 modalità BINNED
1,00
0,90
0,80
0,70
0,60
0,50
0,40
0,30
0,20
0,10
0,00
0,70
0,66
0,88
0,69
0,73
0,96
0,96
0,75
0,76
0,93
Auto
Breast-Cancer
Carseats
Churn
Diabetes
HouseVotes
Iris
Parkinsons
Prostate
Sacramento
MDT
SKL

Segue la tabella riassuntiva delle vittorie, medie e deviazioni standard totali:

	MDT	SKLEARN
MEDIA	0,8414	0,8741
STD	0,0098	0,0042
WINS	9	27

4.3.3 Misuriamo il tempo

	TEMPO DI ESECUZIONE IN MILLISECONDI					
	Classic		Median		Binned	
DATASET	MDT	SKL	MDT	SKL	MDT	SKL
Auto	16666,0521	2,0182	758,7152	1,3468	572,0809	1,5361
Breast-Cancer	160,5158	1,6310	146,2131	1,3640	180,2812	0,9542
Carseats	19999,5730	5,3451	1562,3338	2,3971	863,3039	1,4908
Churn	201342,5062	1,8492	2487,1500	1,8623	2657,8460	1,9426
Diabetes	27856,9472	4,1931	1413,8319	2,3460	1453,5592	1,9462
HouseVotes	144,9978	1,3709	135,6590	1,0567	131,0821	0,8059
Iris	697,1960	4,6749	386,6210	1,9090	214,3610	1,3099
Parkinsons	69797,8599	4,7150	4095,7429	3,1629	2081,2029	3,1097
Prostate	3335,8209	4,3492	780,6048	2,9342	336,3130	2,4190
Sacramento	241112,8542	5,7471	2524,3368	2,7850	956,4180	2,3150
MEDIA	***58111,4323***	***3,5894***	***1429,1208***	***2,1164***	***944,6448***	***1,7829***
STD	***84404,9638***	***1,5931***	***1213,4041***	***0,6878***	***823,4618***	***0,6717***

Figura 4.12: I tempi di esecuzione, in millisecondi, sono stati misurati sulla stessa macchina, nelle stesse condizioni e sugli stessi dataset.

Sebbene il MDT sia in grado, come abbiamo visto nel capitolo precedente, di eguagliare o addirittura, in alcuni casi, di superare le performance dell'albero di Sklearn, quest'ultimo però è stato programmato da esperti per essere iper-efficiente in tempo e spazio.

Nella tabella di Figura 4.12 è possibile notare le differenze enormi di tempi, in millisecondi, di esecuzione tra i due alberi sugli stessi dataset e nelle stesse modalità, ma sopratutto, si può

notare la differenza tra la modalità classic, che genera una colonna per ogni valore unico continuo, e la modalità median, che invece approssima quella classica usando un numero prefissato di bins che quindi andranno a generare molte meno colonne di quella classica mantenendo però, come si è visto nei grafici e nelle tabelle precedenti, a volte la stessa accuracy e F1_SCORE di quella classica. Ne consegue che se si volesse decretare un vincitore interno per il MDT, si potrebbe concludere che trattasi proprio della modalità median.

Se in un particolare contesto è opportuno considerare fattori di tempo e spazio, ad esempio, un dataset contenente molte colonne a valori continui e tantissime righe, allora la prima scelta consisterebbe nella modalità median, ma se il contesto non richiede particolari limiti di tempo o spazio allora la modalità classic è l'unica in grado di fornire la massima ACCURACY o F1_SCORE.

Il MDT non riesce minimamente a competere con quello di Sklearn in termini di complessità, tanto da essergli inferiore di quasi un ordine di grandezza. Però, il supporto nativo per gli attributi categorici e soprattutto il multi-split (ogni nodo radice e interno può avere un numero arbitrario di figli) rendono comunque sensata questa implementazione.

Si afferma il vero quando si parla di inefficienza del MDT in fase di modellizzazione, ma si afferma altrettanto il vero quando si conclude che tale inefficienza può tranquillamente essere tollerata, considerando che il modello si addestra *solo una volta*, mentre, per tutte le restanti indefinite volte, l'albero verrà utilizzato solo per etichettare e classificare dati unlabeled e in questo è ottimamente efficiente tanto quanto lo è quello di sklearn: al caso pessimo la complessità temporale è lineare con la massima profondità dell'albero.

Capitolo 5

Conclusioni

L'obiettivo primario di questa tesi è stato quello di sviluppare ex novo un albero decisionale caratterizzato da una notevole *leggibilità, espressività e versatilità.* È fondamentale sottolineare che l'efficienza computazionale non è stata considerata una priorità. Le caratteristiche fondamentali citate sopra si sono potute ottenere grazie alla creazione di un modello decisionale di tipo decision tree, intrinsecamente categorico e multi-split.

La natura multi-split del MDT agevola in maniera significativa la comprensione umana, poiché, qualora un nodo interno o radice presenti più di due figli, l'utente può osservare numerosi archi uscenti dal nodo stesso; ciascun arco rappresenta un valore categorico unico per l'attributo considerato.

Inoltre, la caratteristica intrinsecamente categorica del MDT consente l'acquisizione di dataset comunemente impiegati nella letteratura scientifica.

Sebbene il MDT non privilegi l'efficienza, esso è in grado di fornire risultati analoghi in termini di accuratezza e F1-score rispetto al Decision Tree di sklearn, il quale, al contrario, enfatizza di molto l'efficienza computazionale. In effetti, l'unico ambito in cui il DT di scikit-learn mostra una chiara superiorità è l'efficienza in termini di tempo e spazio. Tuttavia, l'efficienza non è stata considerata tra gli obiettivi principali di questo progetto, allo stesso modo in cui leggibilità ed espressività non possono essere definite caratteristiche distintive del DT proposto dalla libreria scikit-learn.

Pertanto, è possibile affermare che l'obiettivo della tesi è stato raggiunto con successo. Nelle modalità *classic* e *median*, il MDT dimostra Accuracy ed F1-score generalmente soddi-

sfacenti e, in alcuni casi, addirittura superiori rispetto al DT di sklearn, come osservato nel capitolo precedente.

È importante sottolineare che, sebbene il MDT sia meno efficiente nella fase di modellazione (che avviene una sola volta), nella fase di classificazione (ovvero l'assegnazione di etichette a dati non precedentemente analizzati) risulta tanto efficiente quanto lo è il DT di scikit-learn, dal momento che la loro complessità computazionale è lineare con la profondità dell'albero.

In conclusione, il MDT rappresenta un'opzione estremamente valida, tranne nel caso in cui si debbano gestire dataset di dimensioni particolarmente ampie. L'utilizzo del MDT in tali contesti potrebbe non essere consigliabile, poiché la fase di addestramento potrebbe richiedere tempi e risorse di memoria sproporzionati rispetto alle dimensioni del dataset stesso. Tuttavia, se in futuro si desiderasse migliorare l'efficienza del MDT, ci sarebbero diverse aree che potrebbero essere esplorate:

- Ottimizzazione del codice: Rivedere e ottimizzare il codice per ridurre la complessità computazionale e migliorare l'efficienza in termini di tempo di esecuzione e utilizzo della memoria.

- Parallelizzazione: Sfruttare la parallelizzazione per distribuire il lavoro su più processori o core, migliorando così l'efficienza di addestramento e modellazione.

- Algoritmi di pruning: Implementare algoritmi di pruning per ridurre la complessità dell'albero, migliorando l'efficienza computazionale e riducendo il rischio di overfitting.

- Tecniche di apprendimento incrementale: Esplorare metodi di apprendimento incrementale per addestrare l'albero decisionale in modo più efficiente su grandi dataset, aggiornando il modello man mano che vengono ricevuti nuovi dati.

Appendice A

Il codice della tesi è possibile reperirlo, nella sua totalità, al seguente indirizzo: https://github.com/peppe212/MixedDecisionTree.

In appendice sono forniti un paio di listati di codice che descrivono alcune procedure che hanno permesso la creazione di tabelle e grafici sovrapposti. Tali listati forniscono l'output grazie al quale si sono potute generare tutte le tabelle e gli istogrammi presenti nel quarto capitolo.

Tutti gli snippet che seguono, per maggiore chiarezza e semplicità, riguardano solo la configurazione in cui varia **max_depth** tenendo costanti *min_sample_split* e *min_sample_leaf* dal momento che, per le altre due configurazioni, il codice è molto simile.

```
1
2 def _max_depth_varies(tables, graphs, couple, tree_type, criterion, mode, bins, mss, msl
      ):
3
4   # leggo il nome dei file .csv appartenenti allo stesso dataset su cui andro' ad
        effettuare l'analisi
5   mixed_csv = couple[0]
6   sklearn_csv = couple[1]
7
8   # dataset name:
9   temp_ds_name = re.split('/', couple[0])[1]
10  ds_name = re.split('_', temp_ds_name)[0]
11
12  # distinguo i due alberi
13  if tree_type == 'MixedDecisionTree':
14     df = pd.read_csv(mixed_csv)
15  elif tree_type == 'SklearnDT':
16     df = pd.read_csv(sklearn_csv)
17  else:
```

```
        raise Exception("Wrong Decision Tree name...")
    df.insert(0, 'DATASET', ds_name)
    df.insert(1, 'DT_TYPE', tree_type)

    # qui parte la query
    if mode != 'classic':
        df_queried = df[(df['Tree_Mode'] == mode) & (df['Criterion'] == criterion)
                    & (df['Bins'] == bins) & (df['Min_sample_split'] == mss) &
                    (df['Min_sample_leaf'] == msl)]
    else:
        if bins != 20:
            return
        else:
            bins = 0
            df_queried = df[(df['Tree_Mode'] == mode) & (df['Criterion'] == criterion)
                        & (df['Bins'] == bins) & (df['Min_sample_split'] == mss) &
                        (df['Min_sample_leaf'] == msl)]
        # end_if
    # end_if

    # PARTE RELATIVA ALLE TABELLE:
    if tables:
        # creo le directories in cui salvare le tabelle:
        dir_path = 'Tables/' + ds_name + '/'
        try:
            os.makedirs(dir_path, exist_ok=True)
        except OSError as error:
            print(error)
            print("Path '%s' can not be created" % dir_path)

        table_path = dir_path + 'max_depth_varies.csv'
        df_queried.to_csv(table_path, index=False, header=True, mode='a')
    # end_if

    # PARTE RELATIVA AI GRAFICI:
    if graphs:
        # creo le directories in cui salvare i grafici:
        dir_path = 'Graphs/' + ds_name + '/' + 'max_depth_varies/'
        try:
            os.makedirs(dir_path, exist_ok=True)
        except OSError as error:
            print(error)
            print("Path '%s' can not be created" % dir_path)

        metrics_list = ['ACCURACY', 'F1_SCORE']
```

```
63      for metric in metrics_list:
64          graph_path = dir_path + tree_type + '_' + metric + '_' + criterion + '_' + mode
                + '_' \
65                  + str(bins) + '_' + str(mss) + '_' + str(msl) + '.png'
66
67          x = df_queried['Max_Depth']
68          y = df_queried[metric]
69
70          plt.plot(x, y, label=tree_type, marker='o')
71          plt.xlabel('max_depth')
72          plt.ylabel(metric.lower())
73          plt.savefig(graph_path)
74          plt.close()
75      # end_if
76
77      return
```

Listing A.1: Per ogni dataset abbiamo creato tabelle e grafici

Sulla base delle tabelle e dei grafici creati dallo snippet precedente, si sono potuti generare grafici, questa volta però sovrapposti per mostrare come variano determinati iper-parametri per entrambi gli alberi decisionali considerati in questa tesi.

```
1
2   def graph_overlaps(couple, criterion, _bins, mss, msl):
3       # leggo il nome dei file .csv appartenenti allo stesso dataset su cui andro' ad
            effettuare l'analisi
4       mixed_csv = couple[0]
5       sklearn_csv = couple[1]
6
7       # dataset name:
8       temp_ds_name = re.split('/', couple[0])[1]
9       ds_name = re.split('_', temp_ds_name)[0]
10
11      # distinguo i due alberi:
12      df_mdt = pd.read_csv(mixed_csv)
13      df_sk = pd.read_csv(sklearn_csv)
14
15      # parto con le queries:
16      # si scrive piu' volte sullo stesso plot in modo da sovrapporre le linee del grafico
17      tree_mode_list = ['classic', 'median', 'binned']
18      for mode in tree_mode_list:
19          if mode == 'classic':
20              bins = 0
```

```
21      else:
22          bins = _bins
23      # end_if
24      df_mix_queried = df_mdt[(df_mdt['Tree_Mode'] == mode) & (df_mdt['Criterion'] ==
            criterion)
25                          & (df_mdt['Bins'] == bins) & (df_mdt['Min_sample_split'] == mss) &
26                          (df_mdt['Min_sample_leaf'] == msl)]
27
28      df_sk_queried = df_sk[(df_sk['Tree_Mode'] == mode) & (df_sk['Criterion'] ==
            criterion)
29                        & (df_sk['Bins'] == bins) & (df_sk['Min_sample_split'] == mss) &
30                        (df_sk['Min_sample_leaf'] == msl)]
31
32      x = df_mix_queried['Max_Depth']
33      y = df_mix_queried['ACCURACY']
34      plt.plot(x, y, label='mix-%s' % mode, marker='o')
35
36      x = df_sk_queried['Max_Depth']
37      y = df_sk_queried['ACCURACY']
38      plt.plot(x, y, label='skl-%s' % mode, marker='s')
39  # end_for
40
41  dir_path = 'Overlaps_accuracy/' + ds_name + '/' + 'max_depth_varies/'
42  try:
43      os.makedirs(dir_path, exist_ok=True)
44  except OSError as error:
45      print(error)
46      print("Path '%s' can not be created" % dir_path)
47
48  graph_path = dir_path + 'ACCURACY' + '_' + criterion + '_' + str(_bins) \
49           + '_' + str(mss) + '_' + str(msl) + '.png'
50
51  # qui finalmente creo i grafici sovrapposti (overlapped)
52  plt.xlabel('max_depth')
53  plt.ylabel('ACCURACY'.lower())
54  plt.legend()
55  plt.savefig(graph_path)
56  plt.close()
57
58  return
```

Listing A.2: Procedura che crea i grafici sovrapposti al variare di max_depth

Bibliografia

[1] Riccardo Guidotti, Anna Monreale, Salvatore Ruggieri, Franco Turini, Fosca Giannotti, and Dino Pedreschi. A survey of methods for explaining black box models. *ACM computing surveys (CSUR)*, 51(5):1–42, 2018.

[2] Tom M Mitchell and Tom M Mitchell. *Machine learning*, volume 1. McGraw-hill New York, 1997.

[3] Pang-Ning Tan, Michael Steinbach, and Vipin Kumar. Data mining introduction. *People's Posts and Telecommunications Publishing House, Beijing*, 2006.